

Case study: Project Classification (phân loại và dự đoán tình trạng sức khỏe có bị bệnh tim hay không)

1. Tải Dataset và coi Data Overview

Predicted variable y là "target" trong bảng preview dưới đây. Trong đó giá trị "target" = 1 là có bệnh tim, và "target" = 0 là không có bệnh tim.

```
df = pd.read_csv("../input/heart-disease-uci/heart.csv")
df.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

Sử dụng df.info() để có cái nhìn tổng quan về bộ dữ liệu bao gồm loại dữ liệu, dữ liệu thiếu và số lượng records/rows.

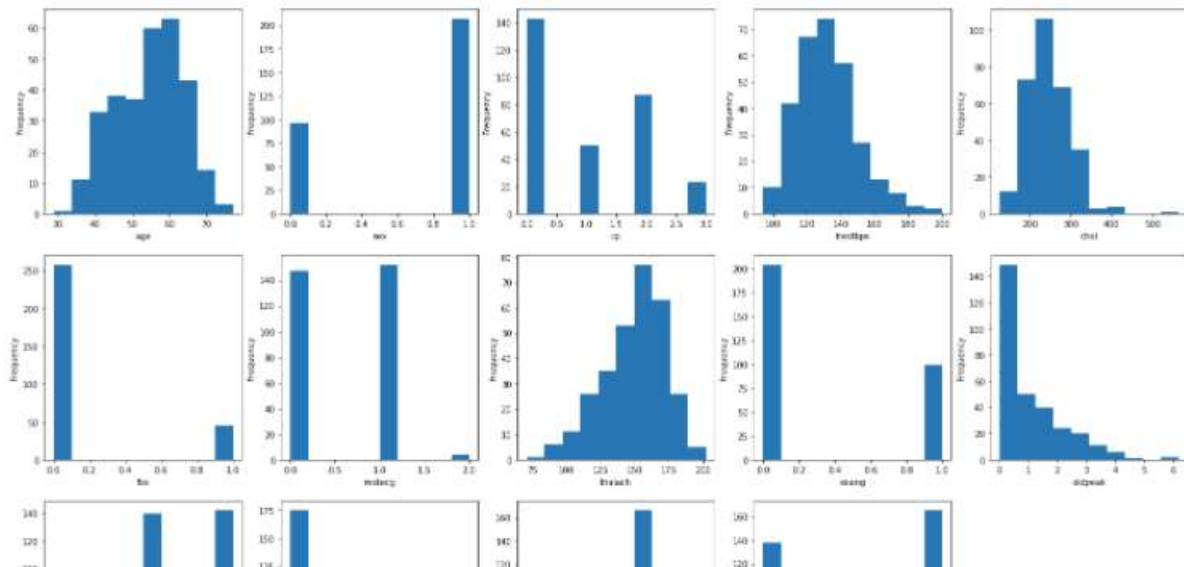
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         303 non-null    int64
1   sex         303 non-null    int64
2   cp          303 non-null    int64
3   trestbps    303 non-null    int64
4   chol        303 non-null    int64
5   fbs         303 non-null    int64
6   restecg     303 non-null    int64
7   thalach     303 non-null    int64
8   exang       303 non-null    int64
9   oldpeak     303 non-null    float64
10  slope       303 non-null    int64
11  ca          303 non-null    int64
12  thal        303 non-null    int64
13  target      303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB

```

2. Exploratory Data Analysis (EDA)

```
fig = plt.figure(figsize=(24, 15))
i = 0
for column in df:
    sub = fig.add_subplot(3, 5, i + 1)
    sub.set_xlabel(column)
    df[column].plot(kind = 'hist')
    i = i + 1
```

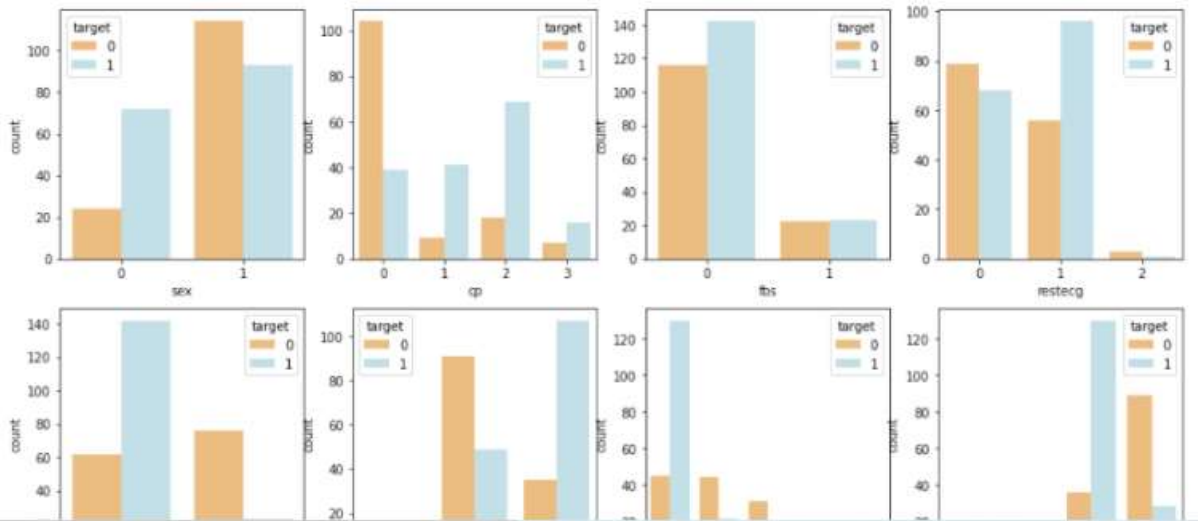


Biểu đồ histogram được sử dụng cho tất cả các numerical features (trong phân tích đơn biến).

Categorical Features vs. Target – Grouped Bar Chart

```
# bar plot
cat_list = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal']
fig = plt.figure(figsize = (16,8))

for i in range(len(cat_list)):
    column = cat_list[i]
    sub = fig.add_subplot(2, 4, i + 1)
    chart = sns.countplot(data = df, x= column, hue= 'target', palette = 'RdYlBu')
```

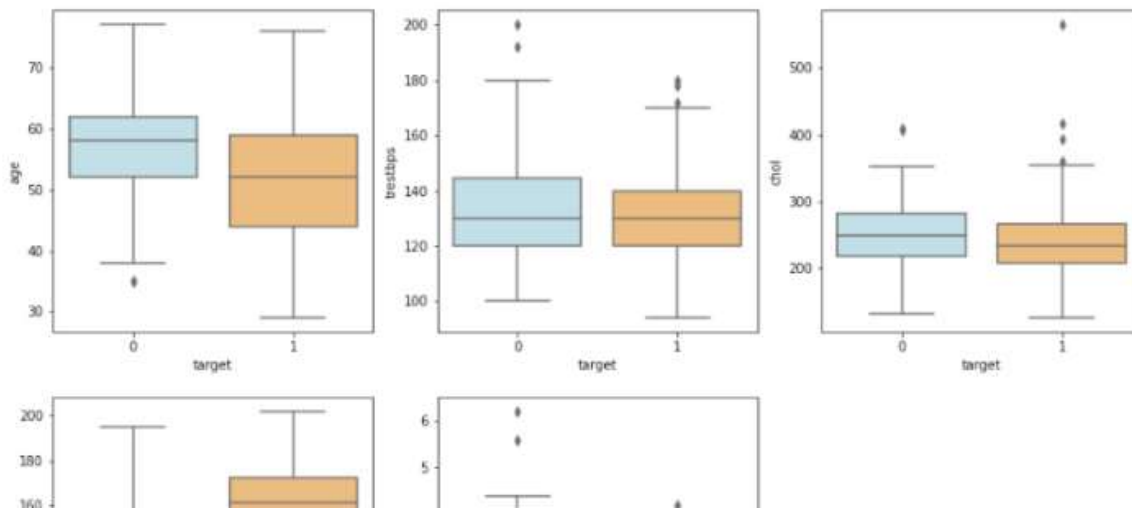


Các dữ liệu categorical được trực quan hoá tương ứng với target (bằng 0 hoặc 1).

Numerical Features vs. Target – Box Plot

```
# box plot
num_list = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']
fig = plt.figure(figsize = (15,10))

for i in range(len(num_list)):
    column = num_list[i]
    sub = fig.add_subplot(2, 3, i + 1)
    sns.boxplot( x = 'target', y = column, data = df, palette = "RdYlBu_r")
```



Box plot được sử dụng để xem tính phân phối của các features.

3. Chia Dataset thành Training & Testing Set

Bộ dữ liệu cần được chia thành một tập con để huấn luyện (train) và một tập con để kiểm tra (test). Mô hình được huấn luyện trên tập huấn luyện và sau đó được kiểm tra bằng tập kiểm tra.

```
from sklearn.model_selection import train_test_split
from sklearn import preprocessing

X = df.drop(['target'], axis=1)
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_st
```

4. Machine Learning Model Pipeline

Để tạo một pipeline (giúp tiết kiệm thời gian và công sức khi phải lặp lại quy trình với các thuật toán khác nhau), chúng ta sẽ thêm trạng thái mặc định của tất cả các thuật toán phân loại đã đề cập ở trên vào danh sách mô hình và sau đó lặp chúng để huấn luyện, kiểm tra, dự đoán và đánh giá.

```
# machine learning model_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB

model_pipeline = []
model_pipeline.append(LogisticRegression(solver='liblinear'))
model_pipeline.append(SVC())
model_pipeline.append(KNeighborsClassifier())
model_pipeline.append(DecisionTreeClassifier())
model_pipeline.append(RandomForestClassifier())
model_pipeline.append(GaussianNB())
```

5. Model Evaluation (đánh giá mô hình)

```
from sklearn import metrics
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix

model_list = ['Logistic Regression', 'SVM', 'KNN', 'Decision Tree', 'Random Forest', 'Naive Bayes']
acc_list = []
auc_list = []
cm_list = []

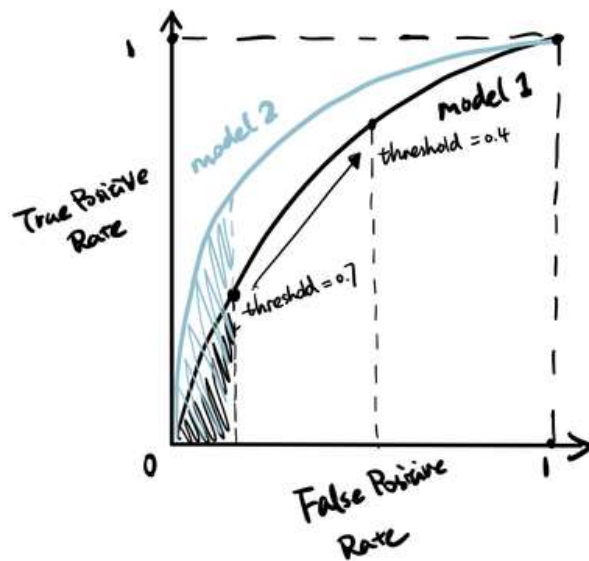
for model in model_pipeline:
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc_list.append(metrics.accuracy_score(y_test, y_pred))
    fpr, tpr, _thresholds = metrics.roc_curve(y_test, y_pred)
    auc_list.append(round(metrics.auc(fpr, tpr), 2))
    cm_list.append(confusion_matrix(y_test, y_pred))
```

Dưới đây là giải thích tóm tắt về các phương pháp đánh giá phổ biến cho các mô hình phân loại — accuracy, ROC & AUC, và confusion matrix.

1. Accuracy

Accuracy là chỉ số đơn giản nhất để đo lường hiệu suất của mô hình. Nó đo tỷ lệ dự đoán chính xác: $accuracy = (true\ positive + true\ negative) / (true\ positive + false\ positive + false\ negative + false\ positive)$.

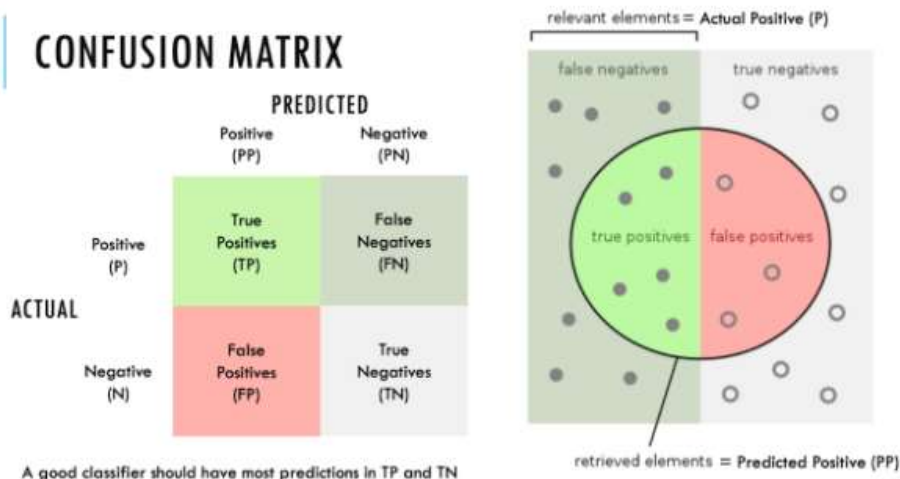
2. ROC & AUC



3. Confusion matrix

Confusion matrix chỉ ra giá trị thực tế so với giá trị dự đoán và tóm tắt các giá trị true negative, false positive, false negative và true positive trong một ma trận.

CONFUSION MATRIX



CÁC CHỈ SỐ QUAN TRỌNG

Tìm hiểu về Confusion matrix trong Machine Learning?

Bài đăng này đã không được cập nhật trong 2 năm

Trong những bài toán phân loại lớp, sau khi đã xử lý dữ liệu và đưa vào mô hình học máy, đầu ra của mô hình sẽ là một vector xác suất tương ứng của từng lớp. Ta có thể đánh giá mức độ chính xác của mô hình thông qua chỉ số về accuracy (độ chính xác): là phần trăm các lớp đã phân loại đúng / tổng số dự đoán, thế nhưng với chỉ số các lớp đã phân loại sai thì sao. Giả sử với bài toán chuẩn đoán ung thư với việc phân loại một người thành 2 lớp: bị ung thư và không bị ung thư. Nếu trong 100 người cần phân loại có 10 người bị ung thư mà mô hình dự đoán cả 100 người đều không bị ung thư thì độ chính xác của mô hình đã là 0.9 rồi, khá cao phải không. Nhưng với 10 người bị ung thư đã bị bỏ qua thì rõ ràng mô hình này không đáng tin cậy chút nào. Cần phải có một phương pháp đánh giá tốt hơn là độ chính xác trong bài toán phân loại và một trong đó là sử dụng Confusion matrix.

Confusion Matrix là gì?

Là một phương pháp đánh giá kết quả của những bài toán phân loại với việc xem xét cả những chỉ số về độ chính xác và độ bao quát của các dự đoán cho từng lớp. Một confusion matrix gồm 4 chỉ số sau đối với mỗi lớp phân loại:

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Để

đơn giản hóa, ta sẽ sử dụng lại bài toán về chẩn đoán ung thư để giải thích 4 chỉ số này. Trong bài toán chẩn đoán ung thư ta có 2 lớp: lớp bị ung thư được chẩn đoán Positive và lớp không bị ung thư được chẩn đoán là Negative:

- TP (True Positive): Số lượng dự đoán chính xác. Là khi mô hình dự đoán đúng một người bị ung thư.
- TN (True Negative): Số lượng dự đoán chính xác một cách gián tiếp. Là khi mô hình dự đoán đúng một người không bị ung thư, tức là việc không chọn trường hợp bị ung thư là chính xác.
- FP (False Positive - Type 1 Error): Số lượng các dự đoán sai lệch. Là khi mô hình dự đoán một người bị ung thư và người đó hoàn toàn khỏe mạnh.
- FN (False Negative - Type 2 Error): Số lượng các dự đoán sai lệch một cách gián tiếp. Là khi mô hình dự đoán một người không bị ung thư nhưng người đó bị ung thư, tức là việc không chọn trường hợp bị ung thư là sai. Từ 4 chỉ số này, ta có 2 con số để đánh giá mức độ tin cậy của một mô hình:
- Precision: Trong tất cả các dự đoán Positive được đưa ra, bao nhiêu dự đoán là chính xác? Chỉ số này được tính theo công thức:

$$\textbf{Precision} = \frac{TP}{TP + FP}$$

- Recall: Trong tất cả các trường hợp Positive, bao nhiêu trường hợp đã được dự đoán chính xác? Chỉ số này được tính theo công thức:

$$\textbf{Recall} = \frac{TP}{TP + FN}$$

Quay trở lại với bài toán chẩn đoán ung thư, giả sử ta có 1 tập dữ liệu gồm 100 người với 90 người khỏe mạnh(Negative) và 10 người mắc bệnh ung thư(Positive) và mô hình dự đoán đúng 2/10 người bị ung thư, tức là đưa ra dự đoán 2 người bị ung thư thì cả 2 dự đoán đều chính xác. Như vậy, chỉ số về Precision khi dự đoán lớp ung thư là 1. Tuy nhiên, 8/10 người còn lại đã bị bỏ qua, từ đó chỉ số về Recall chỉ là 0.2 - một con số rất thấp. Để đánh giá độ tin cậy chung của mô hình, người ta đã kết hợp 2 chỉ số Precision và Recall thành một chỉ số duy nhất: F-score, được tính theo công thức:

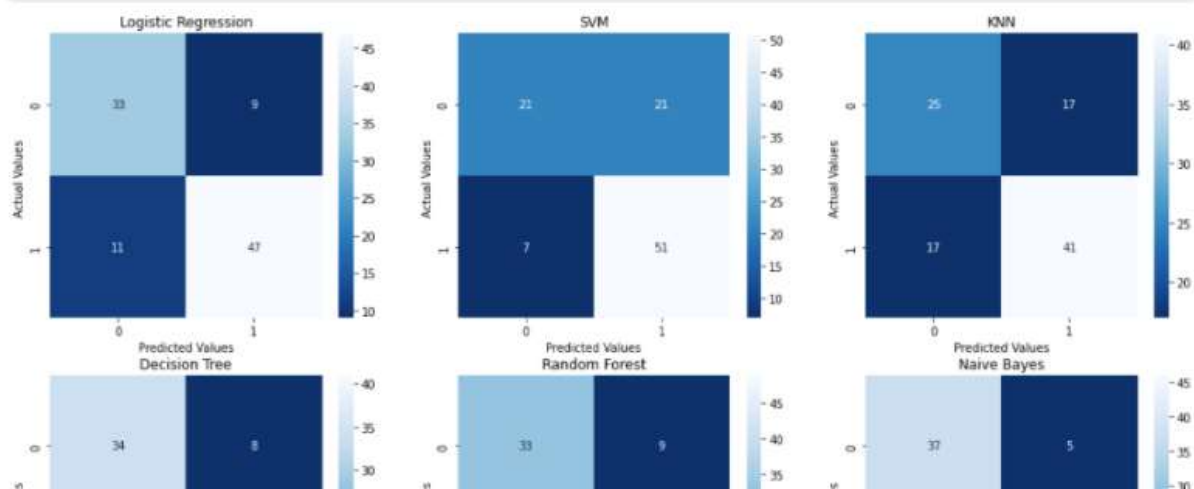
$$\textbf{F - measure} = \frac{2 * \text{Recall} * \text{Precision}}{\text{Recall} + \text{Precision}}$$

Một mô hình có chỉ số F-score cao chỉ khi cả 2 chỉ số Precision và Recall đều cao. Một trong 2 chỉ số này thấp đều sẽ kéo điểm F-score xuống. Trường hợp xấu nhất khi 1 trong hai chỉ số Precision và Recall bằng 0 sẽ kéo điểm F-score về 0. Trường hợp tốt nhất khi cả điểm chỉ số đều đạt giá trị bằng 1, khi đó điểm F-score sẽ là 1.

Qua việc sử dụng chỉ số F-score, ta đã có một thước đo đáng tin cậy về hiệu năng của mô hình trong các bài toán phân loại, đặc biệt khi dữ liệu về một lớp lớn hơn gấp nhiều lần so với dữ liệu về lớp còn lại như trong bài toán chẩn đoán ung thư.

Sau đó, chúng ta có thể sử dụng seaborn để trực quan hóa confusion matrix với biểu đồ heatmap.

```
## plot confusion matrix
fig = plt.figure(figsize = (10,10))
for i in range(len(cm_list)):
    cm = cm_list[i]
    model = model_list[i]
    sub = fig.add_subplot(2, 3, i+1).set_title(model)
    cm_plot = sns.heatmap(cm, annot=True, cmap = 'Blues_r')
    cm_plot.set_xlabel('Predicted Values')
    cm_plot.set_ylabel('Actual Values')
```



```
## accuracy and AUC
result_df = pd.DataFrame({'Model':model_list, 'Accuracy': acc_list, 'AUC': auc_list})
result_df
```

	Model	Accuracy	AUC
0	Logistic Regression	0.80	0.80
1	SVM	0.72	0.69
2	KNN	0.66	0.65
3	Decision Tree	0.75	0.76
4	Random Forest	0.82	0.82
5	Naive Bayes	0.83	0.84

Dựa trên ba phương pháp đánh giá trên, random forest có hiệu suất tốt nhất, trong khi KNN không hoạt động tốt. Tuy nhiên, điều này không có nghĩa là random forest là thuật toán ưu việt hơn. Chúng ta chỉ có thể nói rằng chúng phù hợp hơn với bộ dữ liệu này, nơi kích thước tương đối nhỏ và dữ liệu không cùng tỷ lệ (scale).

Kết luận

Mỗi thuật toán có điểm mạnh/ yếu và yêu cầu các kỹ thuật xử lý dữ liệu khác nhau, ví dụ như KNN nhạy cảm với các features có quy mô (scale) khác nhau, trong khi multicollinearity (các biến độc lập trong một mô hình có mối quan hệ tuyến tính) ảnh hưởng đến kết quả của logistic regression. Nếu bạn muốn hiểu lúc nào nên chọn KNN, lúc nào nên chọn Logistic regression, hay cách chọn các mô hình phù hợp, hãy tham gia khoá học khóa học [Advanced Analytics with Python](#) của [TM Data school](#) với 100% giảng viên được đào tạo về Data science tại nước ngoài (top 1% các chương trình về Data science trên thế giới) nhé. ■

Phân loại thuật toán Machine learning dựa trên ứng dụng

Các thuật toán [Machine Learning](#) còn có thể phân loại dựa trên các ứng dụng hoặc bài toán mà chúng được áp dụng. Dưới đây là một số ứng dụng thường gặp:

1. Nhận dạng hình ảnh và xử lý ảnh

Các thuật toán [Machine Learning](#) được sử dụng rộng rãi trong nhận dạng hình ảnh, như phân loại và nhận dạng đối tượng trong ảnh, chế tác ảnh, phân đoạn ảnh, phát hiện biên báo giao thông, gợi ý bộ lọc ảnh, và nhiều hơn nữa. Một số thuật toán phổ biến trong lĩnh vực này là các mạng Neuron tích chập (Convolutional Neural Network, CNN) và mô hình phân loại đa nhãn (Multi-label [classification](#)).

2. Xử lý ngôn ngữ tự nhiên

Các thuật toán [Machine Learning](#) cũng được ứng dụng trong lĩnh vực [xử lý ngôn ngữ tự nhiên](#) (Natural Language Processing, [NLP](#)), như phân loại văn bản, phân tích cảm xúc, gạch ý chính, dịch máy, tóm tắt văn bản và sinh văn bản tự động. Các mô hình như [BERT](#) và GPT là các kỹ thuật phổ biến được sử dụng trong [xử lý ngôn ngữ tự nhiên](#).

3. Hệ thống đề xuất

4. Âm nhạc và nghệ thuật

[Machine Learning](#) còn được sử dụng để tạo ra các tác phẩm âm nhạc, họa tiết nghệ thuật và kiến trúc tự động. Các thuật toán trong lĩnh vực này bao gồm các mạng [Generative Adversarial](#) (GAN), [Autoencoder](#) và mô hình Markov Hidden.

5. Khoa học, Y tế, Pháp luật và Giáo dục

Thuật toán [Machi](#)

Thuật toán [Machine Learning](#) có nhiều ứng dụng trong các lĩnh vực như Khoa học, Y tế và Pháp luật, nhằm giải quyết các bài toán phức tạp và tối ưu hóa quá trình ra quyết định. Một số ứng dụng phổ biến trong các lĩnh vực này là:

Dự đoán kết quả y tế của bệnh nhân, dựa trên nền tảng của các kỹ thuật phân loại như Random Forest hay mạng Neuron Nhân tạo.

Hiểu và phát hiện các bản án tương tự, hỗ trợ phán quyết của tòa án.

Phát hiện gian lận, bằng cách sử dụng các kỹ thuật như phân cụm K-Means hoặc phân loại Anomaly Detection (phát hiện bất thường).

Tối ưu hoá và nắm bắt xu hướng trong thị trường chứng khoán dựa trên kỹ thuật hồi quy và mô hình học tăng cường (Reinforcement Learning).

Phát hiện và dự đoán các tổn thất hoạt động, bảo trì và sửa chữa của các sản phẩm công nghiệp thông qua việc sử dụng các kỹ thuật dự báo như ARIMA.

Tối ưu hóa chất lượng giáo dục thông qua việc sử dụng mô hình học máy trong việc phát hiện các yếu tố ảnh hưởng đến độ linh hoạt của chương trình học tập, giúp nắm bắt nhu cầu của học viên và xây dựng chương trình học tập phù hợp với từng cá nhân.

Kết luận

Trong bài 2 này, chúng ta đã tìm hiểu về các tiêu chí phân loại các thuật toán [Machine Learning](#) dựa trên cách học, kỹ thuật sử dụng và ứng dụng của chúng trong thực tiễn. Những thông tin trong bài sẽ giúp bạn hiểu rõ hơn về các thuật toán [Machine Learning](#) cũng như cách lựa chọn phù hợp cho từng bài toán cụ thể mà bạn đang giải quyết.

Các bài tiếp theo trong chuỗi [Machine Learning cơ bản](#) của [trituenhantao.io](#) sẽ đi sâu vào các thuật toán và kỹ thuật được giới thiệu ở đây, giúp bạn nắm vững kiến thức và kỹ năng cần thiết khi tiếp cận các bài toán [Machine Learning](#) trong thực tế. Hãy tiếp tục theo dõi và học tập!

Bài Toán Phân Lớp trong Machine Learning - Classification in Machine Learning

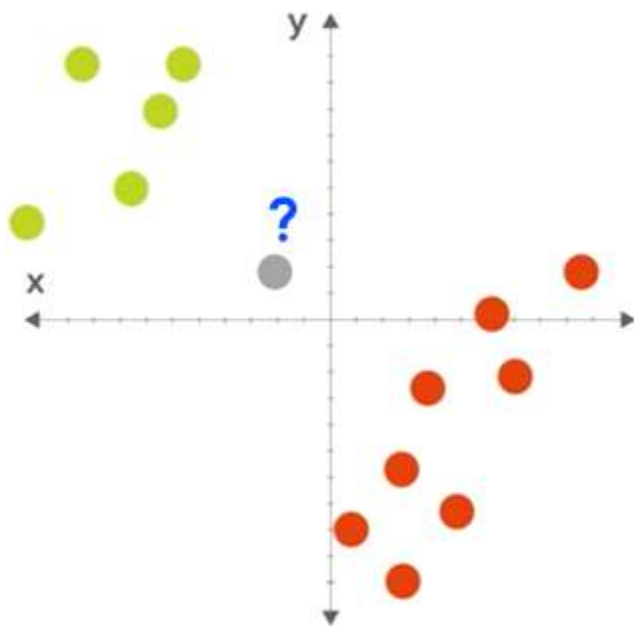
Bài toán phân lớp là quá trình phân lớp 1 đối tượng dữ liệu vào 1 hay nhiều lớp đã cho trước nhờ 1 mô hình phân lớp (model).

Bài toán phân lớp

Bài toán phân lớp là quá trình phân lớp 1 đối tượng dữ liệu vào 1 hay nhiều lớp đã cho trước nhờ 1 mô hình phân lớp (model).

- Mô hình này được xây dựng dựa trên 1 tập dữ liệu được xây dựng trước đó có gán nhãn (hay còn gọi là tập huấn luyện).
- Quá trình phân lớp là quá trình gán nhãn cho đối tượng dữ liệu.

Bài toán phân lớp (classification) và bài toán gom cụm (cluster) là 2 bài toán lớn trong lĩnh vực Machine Learnig (ML).



Chấm xám sẽ thuộc lớp xanh hay đỏ?

Như vậy, nhiệm vụ của bài toán phân lớp là cần tìm 1 **mô hình phân lớp** để khi có dữ liệu mới thì có thể xác định được dữ liệu đó thuộc vào phân lớp nào.

Có nhiều bài toán phân lớp dữ liệu như phân lớp nhị phân (binary classification), phân lớp đa lớp (multiclass classification), phân lớp đa trị.

- Bài toán phân lớp nhị phân là bài toán gán nhãn dữ liệu cho đối tượng vào 1 trong 2 lớp khác nhau dựa vào việc dữ liệu đó có hay không có các đặc trưng (feature) của bộ phân lớp.
- Bài toán phân lớp đa lớp là quá trình phân lớp dữ liệu với số lượng lớp lớn hơn 2. Như vậy với từng dữ liệu phải xem xét và phân lớp chúng vào những lớp khác nhau chứ không phải là 2 lớp như bài toán phân lớp nhị phân. Và thực chất bài toán phân lớp nhị phân là 1 bài toán đặt biệt của phân lớp đa lớp.

Ứng dụng của bài toán này được sử dụng rất nhiều và rộng rãi trong thực tế ví dụ như bài toán nhận dạng khuôn mặt, nhận diện giọng nói, phát hiện email spam, ...



Phân lớp nhận dạng email spam

Và đối với các bài toán phân lớp dữ liệu sử dụng các thuật toán học có giám sát (supervised learning) để xây dựng mô hình cho bài toán này.

Quá trình phân lớp dữ liệu

Để xây dựng được mô hình phân lớp và đánh giá được mô hình phải trải qua các quá trình sau.

1. Chuẩn bị tập dữ liệu huấn luyện (dataset) và rút trích đặc trưng (feature extraction)

Công đoạn này được xem là công đoạn quan trọng trong các bài toán về Machine Learning. Vì đây là input cho việc học để tìm ra mô hình của bài toán. Chúng ta phải biết cần chọn ra những đặc trưng tốt (good feature) của dữ liệu, lược bỏ những đặc trưng không tốt của dữ liệu, gây nhiễu (noise). Ước lượng số chiều của dữ liệu bao nhiêu là tốt hay nói cách khác là chọn bao nhiêu feature. Nếu số chiều quá lớn gây khó khăn cho việc tính toán thì phải giảm số chiều của dữ liệu nhưng vẫn giữ được độ chính xác của dữ liệu (reduce demension).

Ở bước này chúng ta cũng chuẩn bị bộ dữ liệu để test trên mô hình. Thông thường sẽ sử dụng cross-validation (kiểm tra chéo) để chia tập datasets thành 2 phần, 1 phần phục vụ cho training (**training datasets**) và phần còn lại phục vụ cho mục đích testing trên mô hình (**testing dataset**). Có 2 cách thường sử dụng trong cross-validation là **splitting** và **k-fold**.

2. Xây dựng mô hình phân lớp (classifier model)

Mục đích của mô hình huấn luyện là tìm ra hàm $f(x)$ và thông qua hàm f tìm được để gán nhãn cho dữ liệu, bước này thường được gọi là learning hay training.

$$f(x) = y$$

- x : các feature hay input đầu vào của dữ liệu.
- y : nhãn lớp hay output đầu ra.

Thông thường để xây dựng mô hình phân lớp cho bài toán này cần sử dụng các thuật toán học giám sát (supervised learning) như k-nearest neighbors, Neural Network, SVM, Decision tree, Naive Bayes.

3. Kiểm tra dữ liệu với mô hình (making predictions)

Sau khi đã tìm được mô hình phân lớp ở bước 2, thì ở bước này sẽ đưa vào các dữ liệu mới để kiểm tra trên mô hình phân lớp.

4. Đánh giá mô hình phân lớp và chọn ra mô hình tốt nhất

Bước cuối cùng sẽ đánh giá mô hình bằng cách đánh giá mức độ lỗi của dữ liệu testing và dữ liệu training thông qua mô hình tìm được. Nếu không đạt được kết quả mong muốn thì phải thay đổi các tham số (turning parameters) của các thuật toán học để tìm ra các mô hình tốt hơn và kiểm tra, đánh giá lại mô hình phân lớp, và cuối cùng chọn ra mô hình phân lớp tốt nhất cho bài toán.

=====HỒI QUY TUYẾN TÍNH

TẬP DỮ LIỆU

hours of study	Pass
0.2	0
0.3	0
0.4	0
0.5	0
0.6	0
0.8	0
1	0
1.25	0
1.5	0
1.75	0
1.75	1
2	0
2.25	1
2.5	0
2.75	1
3	0
3.25	1

2	0
2.25	1
2.5	0
2.75	1
3	0
3.25	1
3.5	0
4	1
4.25	1
4.5	1
4.75	1
5	1
5.25	1
6	1
6.5	1
6.75	1
7	1

0 LÀ FAIL, 1 LÀ PASS

CỘT 1 SỐ GIỜ HỌC

```
import pandas as pd
import numpy as np
import math
from sklearn.linear_model import LogisticRegression, LinearRegression
import matplotlib.pyplot as plt
from matplotlib.pyplot import figure
```

Logistic Regression with 1 variable

```
filename = "hoursOfStudy.csv"
dataframe = pd.read_csv(filename)
dataframe.columns
#dataframe
```

```
Index(['hours of study', 'Pass'], dtype='object')
```

```
X = dataframe.drop(['Pass'], axis='columns')
X
```

hours of study	
0	0.20
1	0.30
2	0.40
3	0.50
4	0.60
5	0.80
6	1.00
7	1.25

```
In [122]: linear_regression = LinearRegression()
#train this model
linear_regression.fit(X, y)
```

```
Out[122]: LinearRegression()
```

```
In [126]: figure(figsize=(8, 6), dpi=120)
plt.xlabel('Hours of study')
plt.ylabel('Pass/Fail')
#ylim = limit of y
plt.ylim(-0.02, 1.02)
plt.scatter(X, y, marker='+', color='blue')
plt.plot(X, linear_regression.coef_*X + linear_regression.intercept_, color='green')
```

Out[126]: [matplotlib.lines.Line2D at 0x22d3d3cc280]



```
In [122]: linear_regression = LinearRegression()
#train this model
linear_regression.fit(X, y)
```

Out[122]: LinearRegression()

```
In [128]: logistic_regression = LogisticRegression()
#train the model
logistic_regression.fit(X,y)
```

Out[128]: LogisticRegression()

```
In [127]: figure(figsize=(8, 6), dpi=100)
plt.xlabel('Hours of study')
plt.ylabel('Pass/Fail')
#ylim = limit of y
plt.ylim(-0.02, 1.02)
plt.scatter(X, y, marker='+', color='blue')
plt.plot(X, linear_regression.coef_*X + linear_regression.intercept_, color='green')
```

Out[127]: [matplotlib.lines.Line2D at 0x22d3d662430]

```
In [129]: #predict Probability
logistic_regression.predict_proba([[3.1]])
```

#0.39501417+0.60498583 = 1]

C:\Users\sunli\conda\envs\machineLearningTutorial\lib\site-packages\sklearn\base.py:450: UserWarning: X does not have feature names, but LogisticRegression was fitted with feature names
warnings.warn(

Out[129]: array([[0.39501417, 0.60498583]])

Xác suất để xảy ra pass là 0.6049, fail là 0.39501 (2 giá trị này cộng lại sẽ là 1)

Tính cho các điểm khác rồi vẽ đồ thị

```
In [131]: logistic_regression.predict_proba(X)

Out[131]: array([[0.96296722, 0.03703278],
                 [0.95816009, 0.04183991],
                 [0.95275957, 0.04724043],
                 [0.94670075, 0.05329925],
                 [0.93991386, 0.06008614],
                 [0.92385429, 0.07614571],
                 [0.90394106, 0.09605894],
                 [0.87260305, 0.12739695],
                 [0.83293098, 0.16706902],
                 [0.783963, 0.216037],
                 [0.783963, 0.216037],
                 [0.725376, 0.274624],
                 [0.657834, 0.342166],
                 [0.583225, 0.416775],
                 [0.504599, 0.495401]])
```

Hoặc đổi về dataframe

```
Out[129]: array([[0.39501417, 0.60498583]])

In [132]: pd.DataFrame(logistic_regression.predict_proba(X))

Out[132]:
```

	0	1
0	0.962967	0.037033
1	0.958160	0.041840
2	0.952760	0.047240
3	0.946701	0.053299
4	0.939914	0.060086
5	0.923854	0.076146
6	0.903941	0.096059
7	0.872603	0.127397
8	0.832931	0.167069
9	0.783963	0.216037
10	0.783963	0.216037
11	0.725376	0.274624
12	0.657834	0.342166
13	0.583225	0.416775
14	0.504599	0.495401

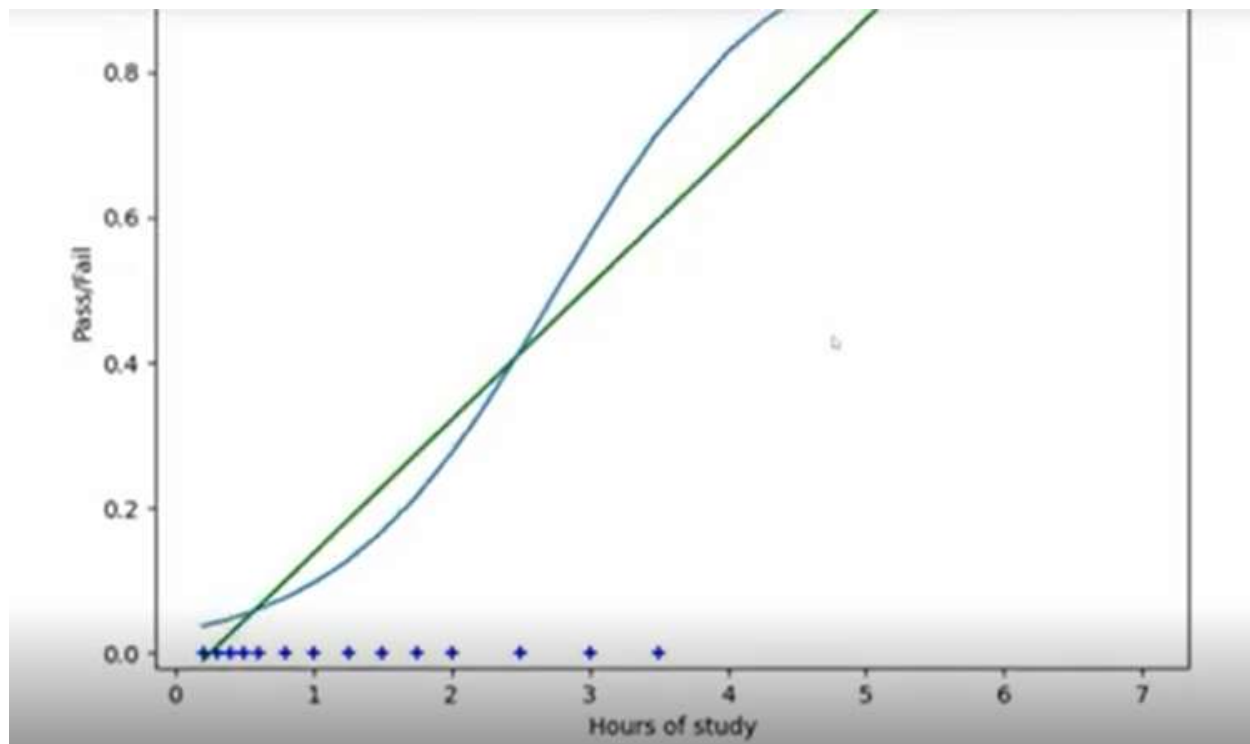
Xác suất để pass

```
In [133]: pd.DataFrame(logistic_regression.predict_proba(X))[1]
```

```
Out[133]: 0    0.037033  
          1    0.041840  
          2    0.047240  
          3    0.053299  
          4    0.060086
```

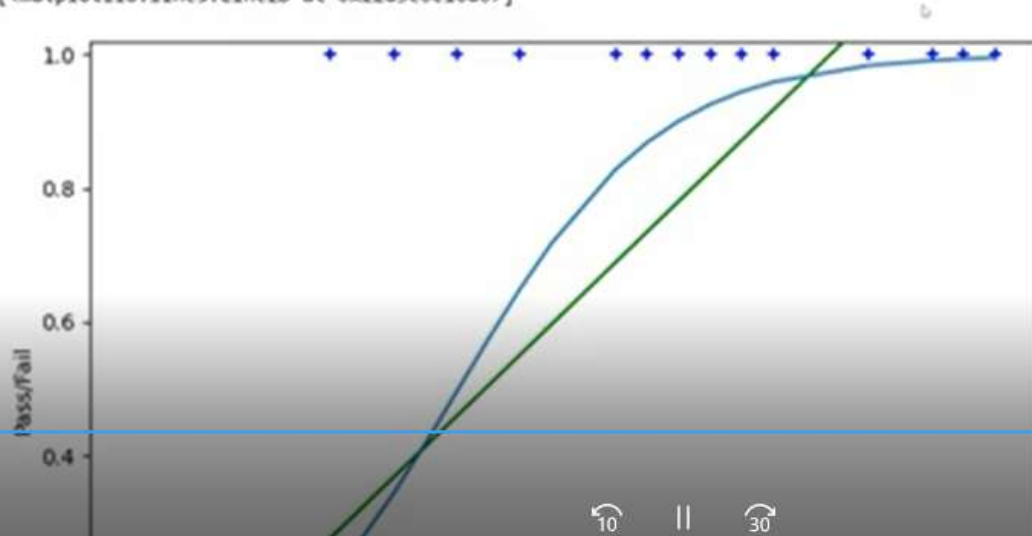
```
In [127]: figure(figsize=(8, 6), dpi=100)  
          plt.xlabel('Hours of study')  
          plt.ylabel('Pass/Fail')  
          #ylim = limit of y  
          plt.ylim(-0.02, 1.02)  
          plt.scatter(X, y, marker='+', color='blue')  
          plt.plot(X, linear_regression.coef_*X + linear_regression.intercept_, color='green')  
          plt.plot(X, pd.DataFrame(logistic_regression.predict_proba(X))[1])
```

```
Out[127]: [<matplotlib.lines.Line2D at 0x22d3d662430>]
```



```
In [134]: figure(figsize=(8, 6), dpi=100)
plt.xlabel('Hours of study')
plt.ylabel('Pass/Fail')
#ylim = limit of y
plt.ylim(-0.02, 1.02)
plt.scatter(X, y, marker='+', color='blue')
plt.plot(X, linear_regression.coef_*X + linear_regression.intercept_, color='green')
plt.plot(X, pd.DataFrame(logistic_regression.predict_proba(X))[1], color='|')
```

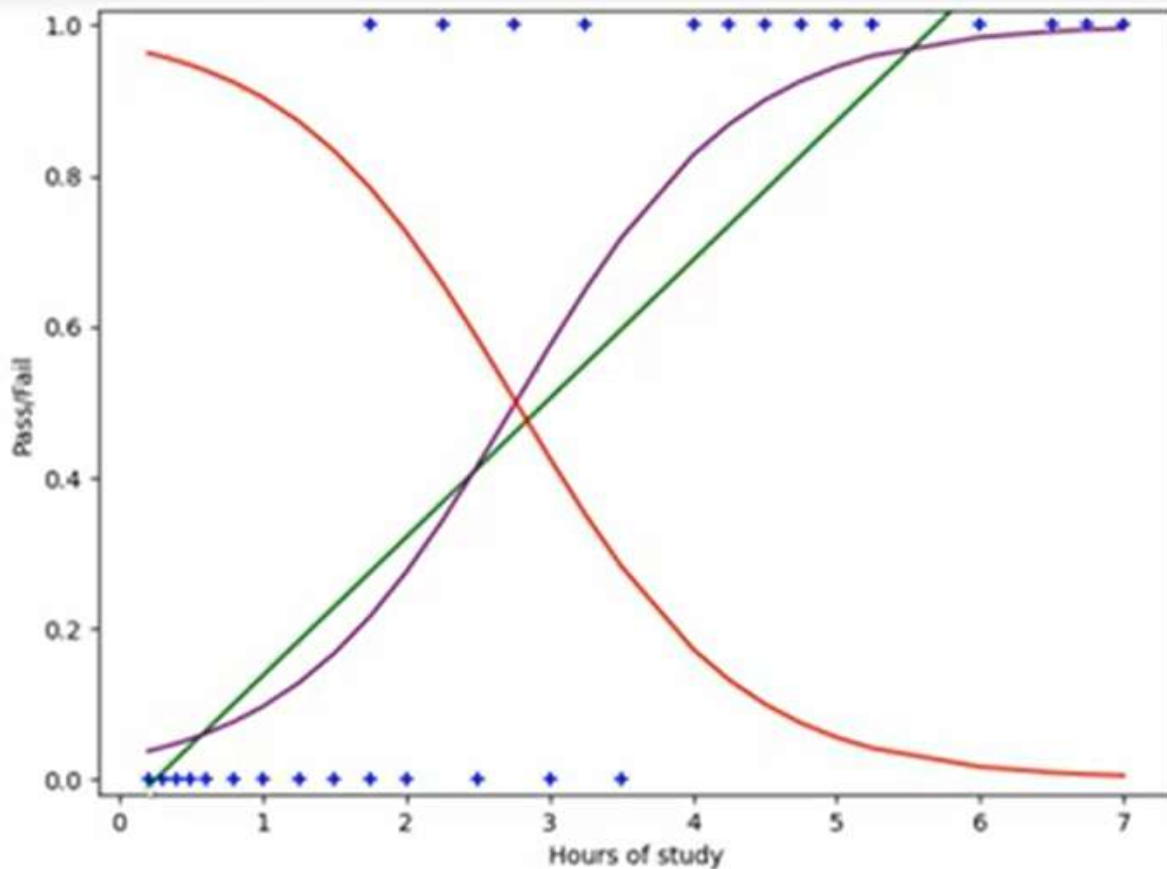
Out[134]: [<matplotlib.lines.Line2D at 0x22d3e0e16d0>]



Color='màu tím'

```
In [135]: figure(figsize=(8, 6), dpi=100)
plt.xlabel('Hours of study')
plt.ylabel('Pass/Fail')
#ylim = limit of y
plt.ylim(-0.02, 1.02)
plt.scatter(X, y, marker='+', color='blue')
plt.plot(X, linear_regression.coef_*X + linear_regression.intercept_, color='green')
plt.plot(X, pd.DataFrame(logistic_regression.predict_proba(X))[1], color='purple')
plt.plot(X, pd.DataFrame(logistic_regression.predict_proba(X))[0], color='red')
```

Out[135]: [<matplotlib.lines.Line2D at 0x22d3e18f190>]



Biểu diễn điểm 3.1 trên đồ thị

```
In [138]: #predict Probability
(logistic_regression.predict_proba([[3.1]]))[0][1]
0.6049858170756617

C:\Users\sun11\conda\envs\machineLearningTutorial\lib\site-packages\sklearn\base.py:450: UserWarning: X does not have valid feature names, but LogisticRegression was fitted with feature names
warnings.warn[

Out[138]: 0.6049858170756617
```

Dùng hàm scatter

```
In [136]: figure(figsize=(8, 6), dpi=100)
plt.xlabel('Hours of study')
plt.ylabel('Pass/Fail')
#ylim = limit of y
plt.ylim(-0.02, 1.02)
plt.scatter(X, y, marker='+', color='blue')
plt.plot(X, linear_regression.coef_*X + linear_regression.intercept_, color='green')
plt.plot(X, pd.DataFrame(logistic_regression.predict_proba(X))[1], color='purple')
plt.plot(X, pd.DataFrame(logistic_regression.predict_proba(X))[0], color='red')
plt.scatter(3.1, (logistic_regression.predict_proba([[3.1]]))[0][1], color='purple', marker='.',)

Out[136]: [matplotlib.lines.Line2D at 0x22d3e1fd60]
```



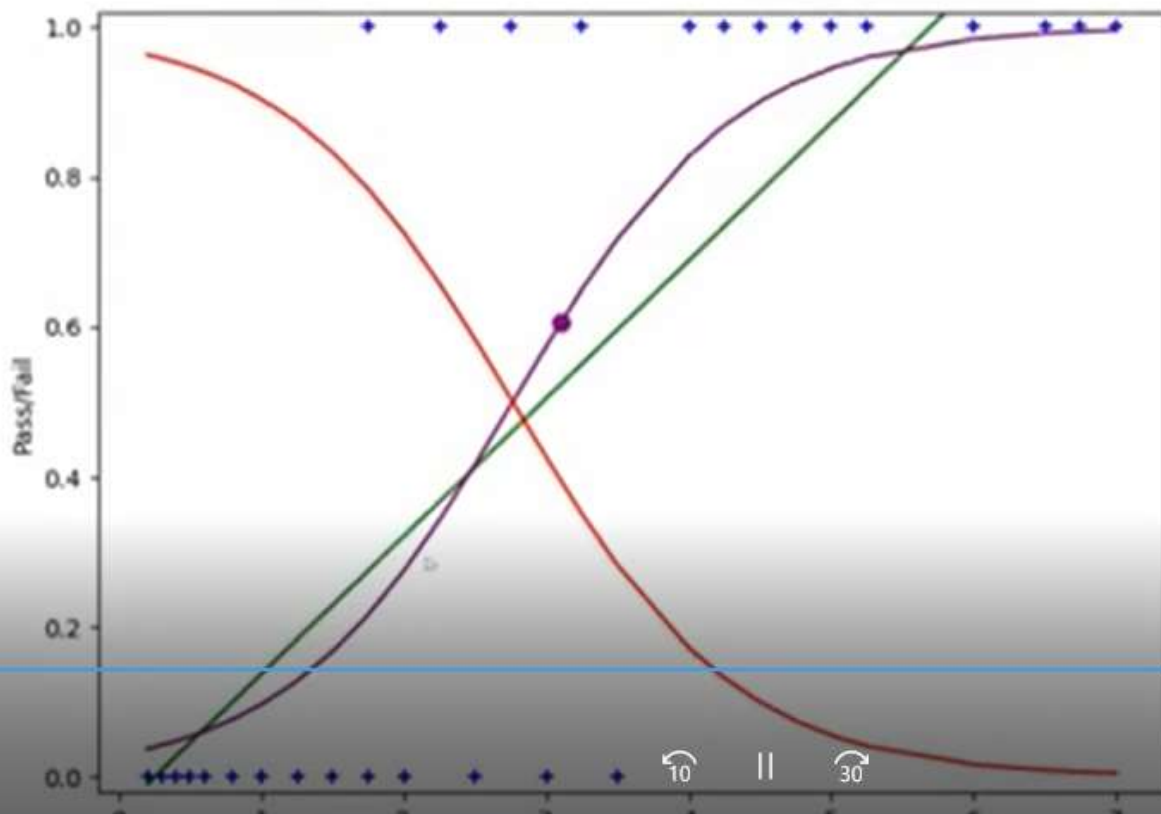
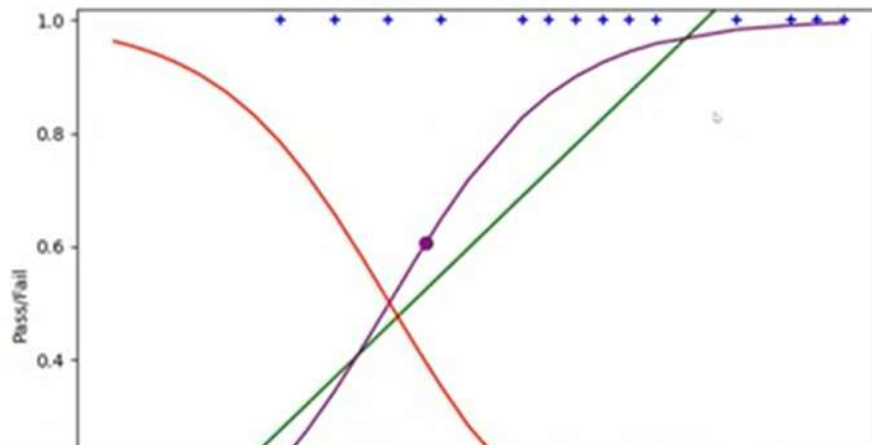
```

plt.plot(X, linear_regression.coef_*X + linear_regression.intercept_, color='green')
plt.plot(X, pd.DataFrame(logistic_regression.predict_proba(X))[1], color='purple')
plt.plot(X, pd.DataFrame(logistic_regression.predict_proba(X))[0], color='red')
plt.scatter(3.1, (logistic_regression.predict_proba([[3.1]]))[0][1], color='purple', marker='.', s=200)

```

C:\Users\sunli\conda\envs\machineLearningTutorial\lib\site-packages\sklearn\base.py:450: UserWarning: X does not have names, but LogisticRegression was fitted with feature names
warnings.warn(

Out[140]: <matplotlib.collections.PathCollection at 0x22d3e5e2820>




```
X = dataframe.drop(['Pass'], axis='columns')
y = dataframe['Pass']
y
```

```
0    0
1    0
2    0
3    0
4    0
5    0
6    0
7    0
8    0
```

```
In [121]: plt.xlabel('Hours of study')
plt.ylabel('Pass/Fail')
plt.scatter(X, y, marker='+', color='blue')
```

```
Out[121]: <matplotlib.collections.PathCollection at 0x22d376b1070>
```



HỒI QUY TUYẾN TÍNH NHIỀU BIẾN

```
In [3]: import pandas as pd
import numpy as np
import math
from sklearn.linear_model import LogisticRegression
import matplotlib.pyplot as plt
from matplotlib.pyplot import figure
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
#show different between "true" value and "predicted" value
from sklearn.metrics import confusion_matrix
```

Logistic Regression with multiple variables

```
In [28]: load_digits()

Out[28]: {'data': array([[ 0.,  0.,  5., ...,  0.,  0.,  0.],
 [ 0.,  0.,  0., ..., 10.,  0.,  0.],
 [ 0.,  0.,  0., ..., 16.,  9.,  0.],
 ...,
 [ 0.,  0.,  1., ...,  6.,  0.,  0.],
 [ 0.,  0.,  2., ..., 12.,  0.,  0.],
 [ 0.,  0., 10., ..., 12.,  1.,  0.]]),
 'target': array([0, 1, 2, ..., 8, 9, 8]),
 'frame': None,
 'feature_names': ['pixel_0_0',
 'pixel_0_1',
 'pixel_0_2',
 'pixel_0_3',
 'pixel_0_4',
 'pixel_0_5',
 'pixel_0_6',
 'pixel_0_7',
 'pixel_1_0',
 'pixel_1_1',
```

Logistic Regression with multiple variables

```
In [30]: digits = load_digits()
dir(digits)

Out[30]: ['DESCR', 'data', 'feature_names', 'frame', 'images', 'target', 'target_names']

In [ ]: 1
```

Logistic Regression with multiple variables

```
In [31]: digits = load_digits()
dir(digits)
digits
```

```
Out[31]: {'data': array([[ 0.,  0.,  5., ...,  0.,  0.,  0.],
 [ 0.,  0.,  0., ..., 10.,  0.,  0.],
 [ 0.,  0.,  0., ..., 16.,  9.,  0.],
 ...,
 [ 0.,  0.,  1., ...,  6.,  0.,  0.],
 [ 0.,  0.,  2., ..., 12.,  0.,  0.],
 [ 0.,  0., 10., ..., 12.,  1.,  0.])),
 'target': array([0, 1, 2, ..., 8, 9, 8]),
 'frame': None,
 'feature_names': ['pixel_0_0',
 'pixel_0_1',
 'pixel_0_2',
 'pixel_0_3',
 'pixel_0_4',
 'pixel_0_5',
 'pixel_0_6',
 ...]}
```

```
In [33]: digits['images']
```

```
Out[33]: array([[ 0.,  0.,  5., ...,  1.,  0.,  0.],
                [ 0.,  0., 13., ..., 15.,  5.,  0.],
                [ 0.,  3., 15., ..., 11.,  8.,  0.],
                ...,
                [ 0.,  4., 11., ..., 12.,  7.,  0.],
                [ 0.,  2., 14., ..., 12.,  0.,  0.],
                [ 0.,  0.,  6., ...,  0.,  0.,  0.]],

               [[ 0.,  0.,  0., ...,  5.,  0.,  0.],
                [ 0.,  0.,  0., ...,  9.,  0.,  0.],
                [ 0.,  0.,  3., ...,  6.,  0.,  0.],
                ...,
                [ 0.,  0.,  1., ...,  6.,  0.,  0.],
                [ 0.,  0.,  1., ...,  6.,  0.,  0.],
                [ 0.,  0.,  0., ..., 10.,  0.,  0.]],

               [[ 0.,  0.,  0., ..., 12.,  0.,  0.],
                [ 0.,  0.,  3., ..., 14.,  0.,  0.],
                [ 0.,  0.,  8., ..., 16.,  0.,  0.],
                ...,
                [ 0.,  9., 16., ...,  0.,  0.,  0.],
                [ 0.,  3., 13., ..., 11.,  5.,  0.],
                [ 0.,  0.,  0., ..., 16.,  9.,  0.]],

               ...,
```

```
In [34]: len(digits['images'])  
         digits['images']
```

```
Out[34]: 1797
```

```
In [35]: len(digits['images'])  
         digits['images'][0]
```

```
Out[35]: array([[ 0.,  0.,  5., 13.,  9.,  1.,  0.,  0.],
                [ 0.,  0., 13., 15., 10., 15.,  5.,  0.],
                [ 0.,  3., 15.,  2.,  0., 11.,  8.,  0.],
                [ 0.,  4., 12.,  0.,  0.,  8.,  8.,  0.],
                [ 0.,  5.,  8.,  0.,  0.,  9.,  8.,  0.],
                [ 0.,  4., 11.,  0.,  1., 12.,  7.,  0.],
                [ 0.,  2., 14.,  5., 10., 12.,  0.,  0.],
                [ 0.,  0.,  6., 13., 10.,  0.,  0.,  0.]])
```

```
In [36]: len(digits['images'])
         digits['images'][0] #8 x 8 matrix
         digits['images'][0].shape
```

```
Out[36]: (8, 8)
```

```
In [37]: len(digits['images'])
         digits['images'][0] #8 x 8 matrix
         #digits['images'][0].shape
         digits['images'][1] #8 x 8 matrix
```

```
Out[37]: array([[ 0.,  0.,  0., 12., 13.,  5.,  0.,  0.],
                [ 0.,  0.,  0., 11., 16.,  9.,  0.,  0.],
                [ 0.,  0.,  3., 15., 16.,  6.,  0.,  0.],
                [ 0.,  7., 15., 16., 16.,  2.,  0.,  0.],
                [ 0.,  0.,  1., 16., 16.,  3.,  0.,  0.],
                [ 0.,  0.,  1., 16., 16.,  6.,  0.,  0.],
                [ 0.,  0.,  1., 16., 16.,  6.,  0.,  0.],
                [ 0.,  0.,  0., 11., 16., 10.,  0.,  0.]])
```

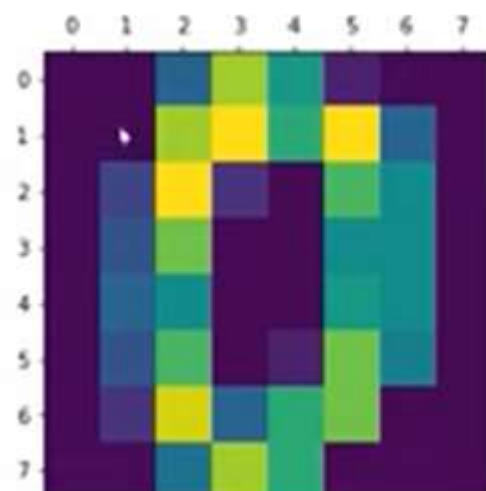
```
In [41]: len(digits['images'])
         #digits['images'][0] #8 x 8 matrix
         #digits['images'][0].shape
         digits['images'][1796] #8 x 8 matrix
```

```
Out[41]: array([[ 0.,  0., 10., 14.,  8.,  1.,  0.,  0.],
                [ 0.,  2., 16., 14.,  6.,  1.,  0.,  0.],
                [ 0.,  0., 15., 15.,  8., 15.,  0.,  0.],
                [ 0.,  0.,  5., 16., 16., 10.,  0.,  0.],
                [ 0.,  0., 12., 15., 15., 12.,  0.,  0.],
                [ 0.,  4., 16.,  6.,  4., 16.,  6.,  0.],
                [ 0.,  8., 16., 10.,  8., 16.,  8.,  0.],
                [ 0.,  1.,  8., 12., 14., 12.,  1.,  0.]])
```

```
Out[41]: array([[ 0.,  0., 10., 14.,  8.,  1.,  0.,  0.],
 [ 0.,  2., 16., 14.,  6.,  1.,  0.,  0.],
 [ 0.,  0., 15., 15.,  8., 15.,  0.,  0.],
 [ 0.,  0.,  5., 16., 16., 10.,  0.,  0.],
 [ 0.,  0., 12., 15., 15., 12.,  0.,  0.],
 [ 0.,  4., 16.,  6.,  4., 16.,  6.,  0.],
 [ 0.,  8., 16., 10.,  8., 16.,  8.,  0.],
 [ 0.,  1.,  8., 12., 14., 12.,  1.,  0.]])
```

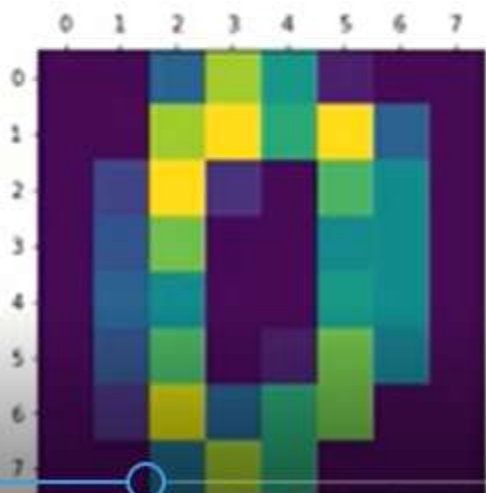
```
In [42]: plt.matshow(digits['images'][0])
```

```
Out[42]: <matplotlib.image.AxesImage at 0x22a5f116dc0>
```



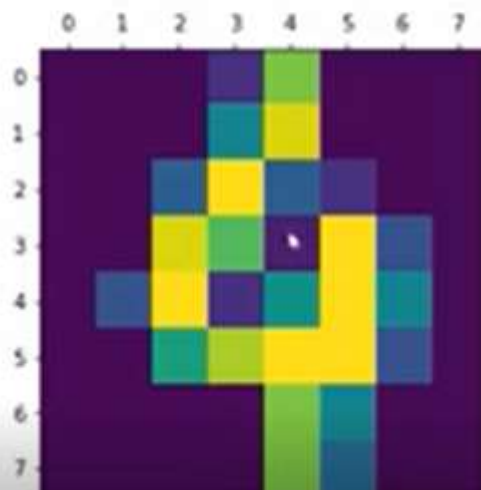
```
In [43]: plt.matshow(digits['images'][0])
         digits['target'][0]
```

```
Out[43]: 0
```



```
In [44]: plt.matshow(digits['images'][100])  
         digits['target'][100]
```

Out[44]: 4



Bài 33: Các phương pháp đánh giá một hệ thống phân lớp

Jan 3, 2018

Trong trang này:

- [1. Giới thiệu](#)
- [2. Accuracy](#)
- [3. Confusion matrix](#)
- [4. True/False Positive/Negative](#)
 - [4.1. True/False Positive/Negative](#)
 - [4.2. Receiver Operating Characteristic curve](#)
 - [4.3. Area Under the Curve](#)
- [5. Precision và Recall](#)
 - [5.1 Định nghĩa](#)
 - [5.2. Precision-Recall curve và Average precision](#)
 - [5.3. F1-score](#)
 - [5.4. Precision-recall cho bài toán phân lớp nhiều lớp](#)
 - [5.4.1. Micro-average](#)
 - [5.4.2. Macro-average](#)
- [6. Tóm tắt](#)
- [7. Tài liệu tham khảo](#)

Bạn có thể download toàn bộ source code dưới dạng Jupyter Notebook [tại đây](#).

1. Giới thiệu

Khi xây dựng một mô hình Machine Learning, chúng ta cần một phép đánh giá để xem mô hình sử dụng có hiệu quả không và để so sánh khả năng của các mô hình. Trong bài viết này, tôi sẽ giới thiệu các phương pháp đánh giá các mô hình classification.

Hiệu năng của một mô hình thường được đánh giá dựa trên tập dữ liệu kiểm thử (test data). Cụ thể, giả sử đầu ra của mô hình khi đầu vào là tập kiểm thử được mô tả bởi vector **y_pred** - là vector dự đoán đầu ra với mỗi phần tử là class được dự đoán của một điểm dữ liệu trong tập kiểm thử. Ta cần so sánh giữa vector dự đoán **y_pred** này với vector class *thật* của dữ liệu, được mô tả bởi vector **y_true**.

Ví dụ với bài toán có 3 lớp dữ liệu được gán nhãn là **0, 1, 2**. Trong bài toán thực tế, các class có thể có nhãn bất kỳ, không nhất thiết là số, và không nhất thiết bắt đầu từ **0**. Chúng ta hãy tạm giả sử các class được đánh số từ **0** đến **C-1** trong trường hợp có **C** lớp dữ liệu. Có 10 điểm dữ liệu trong tập kiểm thử với các nhãn thực sự được mô tả

bởi $y_true = [0, 0, 0, 0, 1, 1, 1, 2, 2, 2]$. Giả sử bộ phân lớp chúng ta đang cần đánh giá dự đoán nhãn cho các điểm này là $y_pred = [0, 1, 0, 2, 1, 1, 0, 2, 1, 2]$.

Có rất nhiều cách đánh giá một mô hình phân lớp. Tùy vào những bài toán khác nhau mà chúng ta sử dụng các phương pháp khác nhau. Các phương pháp thường được sử dụng là: accuracy score, confusion matrix, ROC curve, Area Under the Curve, Precision and Recall, F1 score, Top R error, etc.

Trong Phần 1 này, tôi sẽ trình bày về accuracy score, confusion matrix, ROC curve, và Area Under the Curve. Các phương pháp còn lại sẽ được trình bày trong Phần 2.

2. Accuracy

Cách đơn giản và hay được sử dụng nhất là *accuracy* (độ chính xác). Cách đánh giá này đơn giản tính tỉ lệ giữa số điểm được dự đoán đúng và tổng số điểm trong tập dữ liệu kiểm thử.

Trong ví dụ này, ta có thể đếm được có 6 điểm dữ liệu được dự đoán đúng trên tổng số 10 điểm. Vậy ta kết luận độ chính xác của mô hình là 0.6 (hay 60%). Để ý rằng đây là bài toán với chỉ 3 class, nên độ chính xác nhỏ nhất đã là khoảng 1/3, khi tất cả các điểm được dự đoán là thuộc vào một class nào đó.

```
from __future__ import print_function
import numpy as np

def acc(y_true, y_pred):
    correct = np.sum(y_true == y_pred)
    return float(correct)/y_true.shape[0]

y_true = np.array([0, 0, 0, 0, 1, 1, 1, 2, 2, 2])
y_pred = np.array([0, 1, 0, 2, 1, 1, 0, 2, 1, 2])
print('accuracy = ', acc(y_true, y_pred))

accuracy = 0.6
```

Và đây là [cách tính bằng thư viện](#):

```
from sklearn.metrics import accuracy_score
```

```
print('accuracy = ',accuracy_score(y_true, y_pred))

accuracy = 0.6
```

3. Confusion matrix

Cách tính sử dụng accuracy như ở trên chỉ cho chúng ta biết được bao nhiêu phần trăm lượng dữ liệu được phân loại đúng mà không chỉ ra được cụ thể mỗi loại được phân loại như thế nào, lớp nào được phân loại đúng nhiều nhất, và dữ liệu thuộc lớp nào thường bị phân loại nhầm vào lớp khác. Để có thể đánh giá được các giá trị này, chúng ta sử dụng một ma trận được gọi là *confusion matrix*.

Về cơ bản, confusion matrix thể hiện có bao nhiêu điểm dữ liệu *thực sự* thuộc vào một class, và được *dự đoán* là rơi vào một class. Để hiểu rõ hơn, hãy xem bảng dưới đây:

```
Total: 10 | Predicted | Predicted | Predicted |
          | as: 0 | as: 1 | as: 2 |
-----|-----|-----|-----|---
True: 0 | 2 | 1 | 1 | 4
-----|-----|-----|-----|---
True: 1 | 1 | 2 | 0 | 3
-----|-----|-----|-----|---
True: 2 | 0 | 1 | 2 | 3
-----|-----|-----|-----|---
```

Có tổng cộng 10 điểm dữ liệu. Chúng ta xét ma trận tạo bởi các giá trị tại vùng 3x3 trung tâm của bảng.

Ma trận thu được được gọi là *confusion matrix*. Nó là một ma trận vuông với kích thước mỗi chiều bằng số lượng lớp dữ liệu. Giá trị tại hàng thứ **i**, cột thứ **j** là số lượng điểm **lẽ ra thuộc vào class i nhưng lại được dự đoán là thuộc vào class j**. Như vậy, nhìn vào hàng thứ nhất (**0**), ta có thể thấy được rằng trong số bốn điểm thực sự thuộc lớp **0**, chỉ có hai điểm được phân loại đúng, hai điểm còn lại bị phân loại nhầm vào lớp **1** và lớp **2**.

Chú ý: Có một số tài liệu định nghĩa ngược lại, tức giá trị tại **cột** thứ **i**, **hàng** thứ **j** là số lượng điểm **lẽ ra thuộc vào class i nhưng lại được dự đoán là thuộc vào class j**. Khi đó ta

sẽ được *confusion matrix* là ma trận chuyển vị của *confusion matrix* như cách tôi đang làm. Tôi chọn cách này vì đây chính là cách thư viện *sklearn* sử dụng.

Chúng ta có thể suy ra ngay rằng tổng các phần tử trong toàn ma trận này chính là số điểm trong tập kiểm thử. Các phần tử trên đường chéo của ma trận là số điểm được phân loại đúng của mỗi lớp dữ liệu. Từ đây có thể suy ra *accuracy* chính bằng tổng các phần tử trên đường chéo chia cho tổng các phần tử của toàn ma trận. Đoạn code dưới đây mô tả cách tính *confusion matrix*:

```
def my_confusion_matrix(y_true, y_pred):  
    N = np.unique(y_true).shape[0] # number of classes  
    cm = np.zeros((N, N))  
    for n in range(y_true.shape[0]):  
        cm[y_true[n], y_pred[n]] += 1  
    return cm  
  
cnf_matrix = my_confusion_matrix(y_true, y_pred)  
print('Confusion matrix:')  
print(cnf_matrix)  
print('\nAccuracy:', np.diagonal(cnf_matrix).sum()/cnf_matrix.sum())
```

Confusion matrix:

```
[[ 2.  1.  1.]
```

```
 [ 1.  2.  0.]
```

```
 [ 0.  1.  2.]]
```

Accuracy: 0.6

Cách biểu diễn trên đây của *confusion matrix* còn được gọi là *unnormalized confusion matrix*, tức ma *confusion matrix* chưa chuẩn hoá. Để có cái nhìn rõ hơn, ta có thể dùng *normalized confusion matrix*, tức *confusion matrix* được chuẩn hoá. Để có *normalized confusion matrix*, ta lấy mỗi hàng của *unnormalized confusion matrix* sẽ được chia cho tổng các phần tử trên hàng đó. Như vậy, ta có nhận xét rằng tổng các phần

tử trên một hàng của *normalized confusion matrix* luôn bằng 1. Điều này thường không đúng trên mỗi cột. Dưới đây là cách tính *normalized confusion matrix*:

```
normalized_confusion_matrix = cnf_matrix/cnf_matrix.sum(axis = 1, keepdims = True)
print("\nConfusion matrix (with normalizatrion:)")
print(normalized_confusion_matrix)
Confusion matrix (with normalizatrion:)
[[ 0.5      0.25     0.25    ]
 [ 0.33333333 0.66666667 0.    ]
 [ 0.       0.33333333 0.66666667]]
```

Và cách tính [sử dụng thư viện](#):

```
from sklearn.metrics import confusion_matrix
cnf_matrix = confusion_matrix(y_true, y_pred)
print('Confusion matrix:')
print(cnf_matrix)
Confusion matrix:
[[2 1 1]
 [1 2 0]
 [0 1 2]]
```

Confusion matrix thường được minh hoạ bằng màu sắc để có cái nhìn rõ ràng hơn. Đoạn code dưới đây giúp hiển thị confusion matrix ở cả hai dạng (Nguồn: [Confusion matrix](#)):

```
import matplotlib.pyplot as plt
import itertools
def plot_confusion_matrix(cm, classes,
                           normalize=False,
                           title='Confusion matrix',
                           cmap=plt.cm.Blues):
```

```
"""
```

This function prints and plots the confusion matrix.

Normalization can be applied by setting `normalize=True`.

```
"""
```

```
if normalize:
```

```
    cm = cm.astype('float') / cm.sum(axis=1, keepdims = True)
```

```
plt.imshow(cm, interpolation='nearest', cmap=cmap)
```

```
plt.title(title)
```

```
plt.colorbar()
```

```
tick_marks = np.arange(len(classes))
```

```
plt.xticks(tick_marks, classes, rotation=45)
```

```
plt.yticks(tick_marks, classes)
```

```
fmt = '.2f' if normalize else 'd'
```

```
thresh = cm.max() / 2.
```

```
for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
```

```
    plt.text(j, i, format(cm[i, j], fmt),
```

```
            horizontalalignment="center",
```

```
            color="white" if cm[i, j] > thresh else "black")
```

```
plt.tight_layout()
```

```
plt.ylabel("True label")
```

```
plt.xlabel("Predicted label")
```

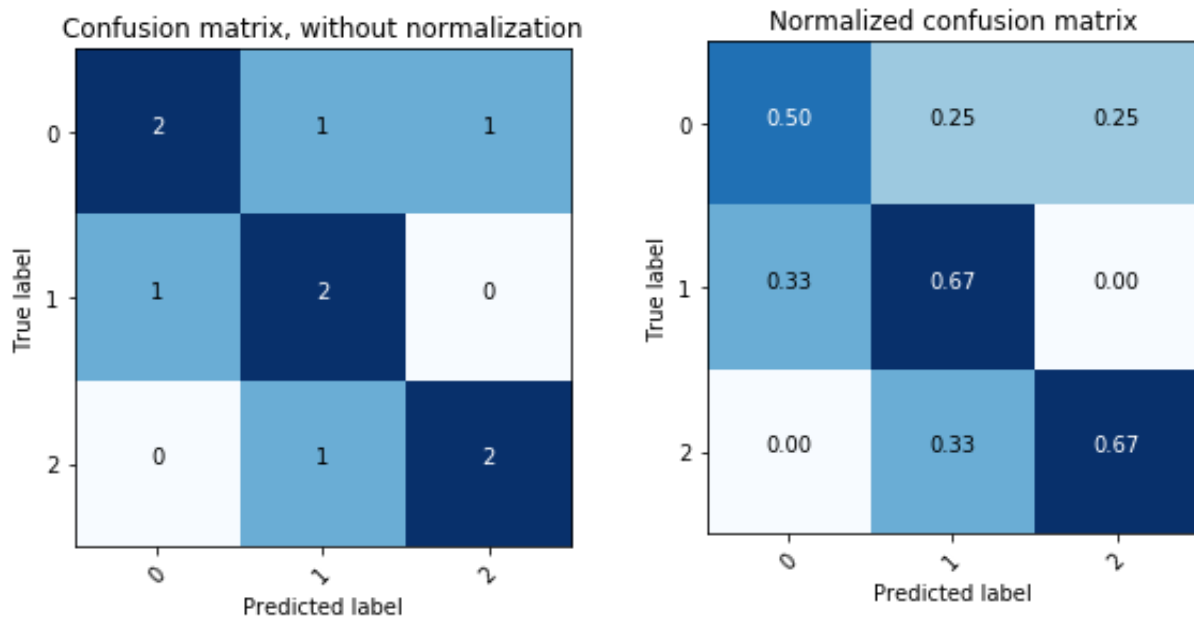
```
# Plot non-normalized confusion matrix
```

```
class_names = [0, 1, 2]
```

```
plt.figure()
plot_confusion_matrix(cnf_matrix, classes=class_names,
                      title='Confusion matrix, without normalization')

# Plot normalized confusion matrix
plt.figure()
plot_confusion_matrix(cnf_matrix, classes=class_names, normalize=True,
                      title='Normalized confusion matrix')

plt.show()
```



Hình 1: Minh hoạ *unnormalized confusion matrix* và *normalized confusion matrix*.

Với các bài toán với nhiều lớp dữ liệu, cách biểu diễn bằng màu này rất hữu ích. Các ô màu đậm thể hiện các giá trị cao. Một mô hình tốt sẽ cho một confusion matrix có các phần tử trên đường chéo chính có giá trị lớn, các phần tử còn lại có giá trị nhỏ. Nói cách khác, khi biểu diễn bằng màu sắc, đường chéo có màu càng đậm so với phần còn lại sẽ càng tốt. Từ hai hình trên ta thấy rằng confusion matrix đã chuẩn hoá mang nhiều thông tin hơn. Sự khác nhau được thấy ở ô trên cùng bên trái. Lớp dữ liệu 0 được phân loại không thực sự tốt nhưng trong *unnormalized confusion matrix*, nó vẫn có màu đậm như hai ô còn lại trên đường chéo chính.

4. True/False Positive/Negative

4.1. True/False Positive/Negative

Cách đánh giá này thường được áp dụng cho các bài toán phân lớp có hai lớp dữ liệu. Cụ thể hơn, trong hai lớp dữ liệu này có một lớp *ng nghiêm trọng* hơn lớp kia và cần được dự đoán chính xác. Ví dụ, trong bài toán xác định có bệnh ung thư hay không thì việc không bị sót (miss) quan trọng hơn là việc chẩn đoán nhầm *âm tính* thành *dương tính*. Trong bài toán xác định có mìn dưới lòng đất hay không thì việc *bỏ sót* nghiêm trọng hơn việc *báo động nhầm* rất nhiều. Hay trong bài toán lọc email rác thì việc cho nhầm email quan trọng vào thùng rác nghiêm trọng hơn việc xác định một email rác là email thường.

Trong những bài toán này, người ta thường định nghĩa lớp dữ liệu *quan trọng* hơn cần được xác định đúng là lớp *Positive* (P-dương tính), lớp còn lại được gọi là *Negative* (N-âm tính). Ta định nghĩa *True Positive (TP)*, *False Positive (FP)*, *True Negative (TN)*, *False Negative (FN)* dựa trên *confusion matrix* chưa chuẩn hoá như sau:

	Predicted	Predicted	
	as Positive	as Negative	
----- ----- -----			
Actual: Positive	True Positive (TP)	False Negative (FN)	
----- ----- -----			
Actual: Negative	False Positive (FP)	True Negative (TN)	
----- ----- -----			

Người ta thường quan tâm đến TPR, FNR, FPR, TNR (R - Rate) dựa trên *normalized confusion matrix* như sau:

	Predicted	Predicted	
--	-----------	-----------	--

	as Positive	as Negative	
Actual: Positive	$TPR = TP/(TP + FN)$	$FNR = FN/(TP + FN)$	
Actual: Negative	$FPR = FP/(FP + TN)$	$TNR = TN/(FP + TN)$	

False Positive Rate còn được gọi là *False Alarm Rate* (tỉ lệ báo động nhầm), *False Negative Rate* còn được gọi là *Miss Detection Rate* (tỉ lệ bỏ sót). Trong bài toán dò mìn, *thà báo nhầm còn hơn bỏ sót*, tức là ta có thể chấp nhận *False Alarm Rate* cao để đạt được *Miss Detection Rate* thấp.

Chú ý::

- Việc biết một cột của confusion matrix này sẽ suy ra được cột còn lại vì tổng các hàng luôn bằng 1 và chỉ có hai lớp dữ liệu.
- **Với các bài toán có nhiều lớp dữ liệu**, ta có thể xây dựng bảng True/False Positive/Negative cho **mỗi lớp** nếu coi lớp đó là lớp *Positive*, các lớp còn lại gộp chung thành lớp *Negative*, giống như cách làm trong [one-vs-rest](#). Bạn có thể xem thêm ví dụ [tại đây](#).

4.2. Receiver Operating Characteristic curve

Trong một số bài toán, việc tăng hay giảm FNR, FPR có thể được thực hiện bằng việc thay đổi một *ngưỡng* (threshold) nào đó. Lấy ví dụ khi ta sử dụng thuật toán [Logistic Regression](#), đầu ra của mô hình có thể là các *lớp cứng* 0 hay 1, hoặc cũng có thể là các giá trị thể hiện xác suất để dữ liệu đầu vào thuộc vào lớp 1. Khi sử dụng thư viện [sklearn Logistic Regression](#), ta có thể lấy được các giá trị xác suất này bằng phương thức `predict_proba()`. Mặc định, ngưỡng được sử dụng là 0.5, tức là một điểm dữ liệu **x** sẽ được dự đoán rơi vào lớp 1 nếu giá trị `predict_proba(x)` lớn hơn 0.5 và ngược lại.

Nếu bây giờ ta coi lớp 1 là lớp *Positive*, lớp 0 là lớp *Negative*, câu hỏi đặt ra là làm thế nào để tăng mức độ *báo nhầm* (FPR) để giảm mức độ *bỏ sót* (FNR)? Chú ý rằng tăng FNR đồng nghĩa với việc giảm TPR vì tổng của chúng luôn bằng 1.

Một kỹ thuật đơn giản là ta thay giá trị threshold từ 0.5 xuống một số nhỏ hơn. Chẳng hạn nếu chọn threshold = 0.3, thì mọi điểm được dự đoán có xác suất đầu ra lớn hơn 0.3 sẽ được dự đoán là thuộc lớp Positive. Nói cách khác, tỉ lệ các điểm được phân loại là Positive sẽ tăng lên, kéo theo cả False Positive Rate và True Positive Rate cùng tăng lên (cột thứ nhất trong ma trận tăng lên). Từ đây suy ra cả FNR và TNR đều giảm.

Scores:

```
[ 0.16987517 0.27608323 0.10851568 0.13395249 0.24878687 0.29100097
 0.21036182 0.48215779 0.01930099 0.30927599 0.26581374 0.15141354
 0.26298063 0.10405583 0.30773121 0.39830016 0.04868077 0.17290186
 0.28717646 0.3340749 0.4174846 0.27292017 0.68740357 0.62108568
 0.20781968 0.43056031 0.67816027 0.47037842 0.23118192 0.68862749
 0.24559788 0.58645887 0.69637251 0.5247967 0.24265087 0.60485646
 0.54800088 0.69565411 0.20509934 0.39638029 0.30860676 0.6267616
 0.42360257 0.5507021 0.50313701 0.67614457 0.60108083 0.25201502
 0.27830655 0.58669514]
```

Nhìn chung, các điểm thuộc lớp **1** có **score** cao hơn. Thư viện sklearn sẽ giúp chúng ta tính các thresholds cũng như FPR và TPR tương ứng:

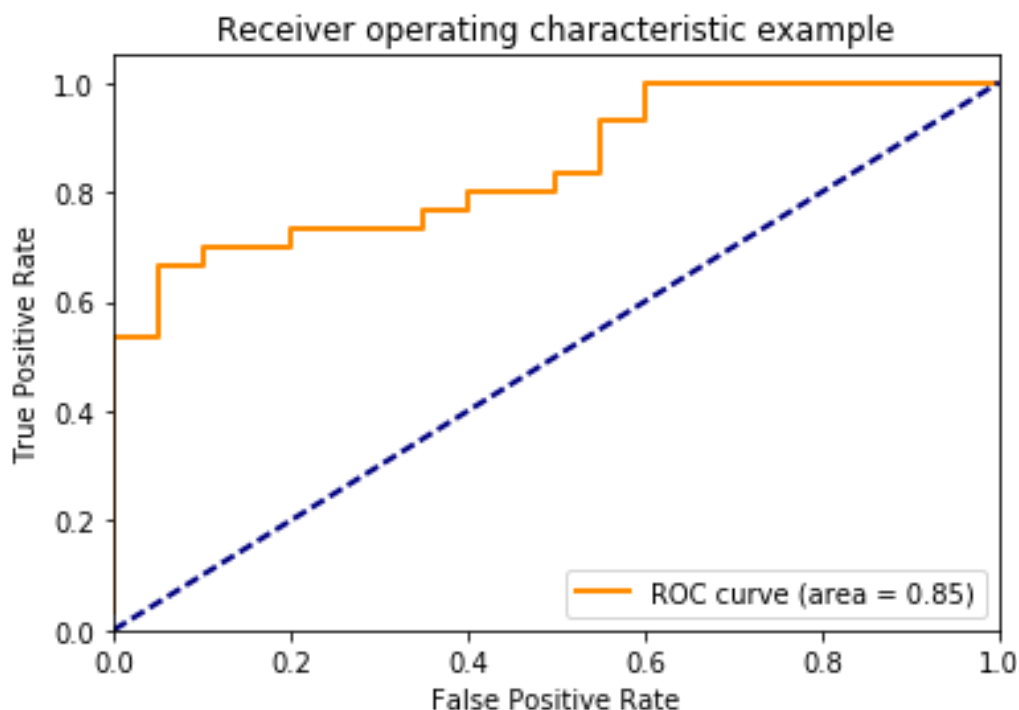
```
from sklearn.metrics import roc_curve, auc
fpr, tpr, thresholds = roc_curve(y_true, scores, pos_label = 1)
print("Thresholds:")
print(thresholds)
Thresholds:
[ 0.69637251 0.50313701 0.48215779 0.4174846 0.39830016 0.39638029
 0.30927599 0.30860676 0.28717646 0.27830655 0.27608323 0.27292017
 0.26298063 0.25201502 0.24878687 0.23118192 0.21036182 0.20509934
 0.01930099]
print('False Positive Rate:')
print(fpr)
False Positive Rate:
[ 0.  0.  0.05 0.05 0.1  0.1  0.2  0.2  0.35 0.35 0.4  0.4
 0.5 0.5 0.55 0.55 0.6 0.6  1. ]
```

```
print("True Positive Rate:")
tpr
True Positive Rate:
array([ 0.03333333, 0.53333333, 0.53333333, 0.66666667, 0.66666667,
        0.7      , 0.7      , 0.73333333, 0.73333333, 0.76666667,
        0.76666667, 0.8      , 0.8      , 0.83333333, 0.83333333,
        0.93333333, 0.93333333, 1.      , 1.      ])
```

Như vậy, ứng với **threshold = 0.69637251**, **fpr = 0** và **tpr = 0.03**. Đây không phải là một ngưỡng tốt vì mặc dù False Positive Rate thấp, True Positive Rate cũng rất thấp. Chúng ta luôn muốn rằng FPR thấp và TPR cao.

ROC cho bài toán này được minh hoạ như dưới đây:

```
import matplotlib.pyplot as plt
from itertools import cycle
plt.figure()
lw = 2
plt.plot(fpr, tpr, color='darkorange',
         lw=lw, label='ROC curve (area = %0.2f)' % auc(fpr, tpr))
plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic example')
plt.legend(loc="lower right")
plt.show()
```



Hình 2: Ví dụ về Receiver Operating Characteristic curve và Area Under the Curve.

4.3. Area Under the Curve

Dựa trên ROC curve, ta có thể chỉ ra rằng một mô hình có hiệu quả hay không. Một mô hình hiệu quả khi có FPR thấp và TPR cao, tức tồn tại một điểm trên ROC curve gần với điểm có tọa độ (0, 1) trên đồ thị (góc trên bên trái). Curve càng gần thì mô hình càng hiệu quả.

Có một thông số nữa dùng để đánh giá mà tôi đã sử dụng ở trên được gọi là *Area Under the Curve* hay *AUC*. Đại lượng này chính là diện tích nằm dưới ROC curve màu cam. Giá trị này là một số dương nhỏ hơn hoặc bằng 1. Giá trị này càng lớn thì mô hình càng tốt.

Chú ý: [Cross validation](#) cũng có thể được thực hiện bằng cách xác định ROC curve và AUC lên [validation set].

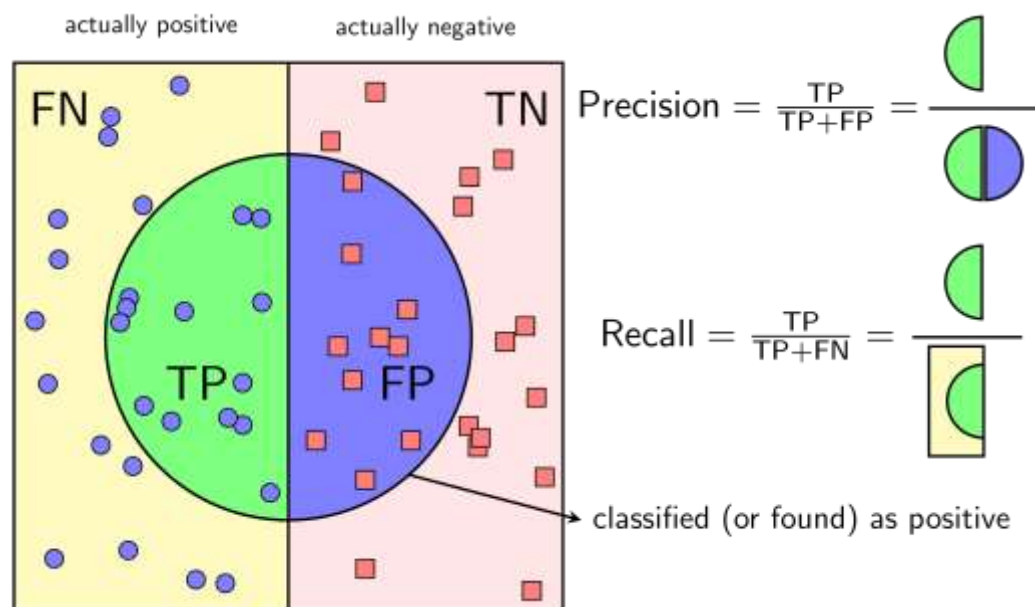
5. Precision và Recall

5.1 Định nghĩa

Với bài toán phân loại mà tập dữ liệu của các lớp là chênh lệch nhau rất nhiều, có một phép đo hiệu quả thường được sử dụng là Precision-Recall.

Trước hết xét bài toán phân loại nhị phân. Ta cũng coi một trong hai lớp là *positive*, lớp còn lại là *negative*.

Xét Hình 3 dưới đây:



Hình 3: Cách tính Precision và Recall.

Với một cách xác định một lớp là *positive*, **Precision** được định nghĩa là tỉ lệ số điểm **true positive** trong số những điểm **được phân loại là positive** (TP + FP).

Recall được định nghĩa là tỉ lệ số điểm **true positive** trong số những điểm **thực sự là positive** (TP + FN).

Một cách toán học, Precision và Recall là hai phân số có tử số bằng nhau nhưng mẫu số khác nhau:

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

Bạn đọc có thể nhận thấy rằng TPR và Recall là hai đại lượng bằng nhau. Ngoài ra, cả Precision và Recall đều là các số không âm nhỏ hơn hoặc bằng một.

Precision cao đồng nghĩa với việc độ chính xác của các điểm tìm được là cao. Recall cao đồng nghĩa với việc True Positive Rate cao, tức tỉ lệ bỏ sót các điểm thực sự *positive* là thấp.

Ví dụ nhỏ dưới đây thể hiện cách tính Precision và Recall dựa vào Confusion Matrix cho bài toán phân loại nhị phân.

```
from __future__ import print_function

import numpy as np

# confusion matrix to precision + recall

def cm2pr_binary(cm):

    p = cm[0,0]/np.sum(cm[:,0])

    r = cm[0,0]/np.sum(cm[0])

    return (p, r)

# example of a confusion matrix for binary classification problem

cm = np.array([[100., 10], [20, 70]])

p,r = cm2pr_binary(cm)

print("precision = {0:.2f}, recall = {1:.2f}".format(p, r))

precision = 0.83, recall = 0.91
```

Khi Precision = 1, mọi điểm tìm được đều thực sự là *positive*, tức không có điểm *negative* nào lẫn vào kết quả. Tuy nhiên, Precision = 1 không đảm bảo mô hình là tốt, vì câu hỏi đặt ra là liệu mô hình đã tìm được tất cả các điểm *positive* hay chưa. Nếu một mô hình chỉ tìm được đúng một điểm *positive* mà nó chắc chắn nhất thì ta không thể gọi nó là một mô hình tốt.

Khi Recall = 1, mọi điểm *positive* đều được tìm thấy. Tuy nhiên, đại lượng này lại không đo liệu có bao nhiêu điểm *negative* bị lẫn trong đó. Nếu mô hình phân loại mọi điểm là *positive* thì chắc chắn Recall = 1, tuy nhiên dễ nhận ra đây là một mô hình cực tồi.

Một mô hình phân lớp tốt là mô hình có cả Precision và Recall đều cao, tức càng gần một càng tốt. Có hai cách đo chất lượng của bộ phân lớp dựa vào Precision và Recall: Precision-Recall curve và F-score.

5.2. Precision-Recall curve và Average precision

Tương tự như ROC curve, chúng ta cũng có thể đánh giá mô hình dựa trên việc thay đổi một ngưỡng và quan sát giá trị của Precision và Recall. Khái niệm Area Under the Curve (AUC) cũng được định nghĩa tương tự. Với Precision-Recall Curve, AUC còn có một tên khác là **Average precision (AP)**.

$$AP = \sum_n (R_n - R_{n-1}) P_n$$

Giả sử có N ngưỡng để tính precision và recall, với mỗi ngưỡng cho một cặp giá trị precision, recall là $P_n, R_n, n = 1, 2, \dots, N$. Precision-Recall curve được vẽ bằng cách vẽ từng điểm có tọa độ (R_n, P_n) trên trục tọa độ và nối chúng với nhau. AP được xác định bằng:

$$AP = \sum_n (R_n - R_{n-1}) P_n$$

ở đó $(R_n - R_{n-1}) P_n$ chính là diện tích hình chữ nhật có chiều rộng $(R_n - R_{n-1})$ và chiều cao P_n , đây cũng gần với cách tính tích phân dựa trên cách tính diện tích của từng hình chữ nhật nhỏ. (Nếu bạn đọc còn nhớ khái niệm *diện tích hình thang cong* thì sẽ tưởng tượng ra.)

Giả sử có N ngưỡng để tính precision và recall, với mỗi ngưỡng cho một cặp giá trị precision, recall là $P_n, R_n, n = 1, 2, \dots, N$. Precision-Recall curve được vẽ bằng cách vẽ từng điểm có tọa độ (R_n, P_n) trên trục tọa độ và nối chúng với nhau. AP được xác định bằng: $AP = \sum_n (R_n - R_{n-1}) P_n$
ở đó $(R_n - R_{n-1}) P_n$ chính là diện tích hình chữ nhật có chiều rộng $(R_n - R_{n-1})$ và chiều cao P_n , đây cũng gần với cách tính tích phân dựa trên cách tính diện tích của từng hình chữ nhật nhỏ. (Nếu bạn đọc còn nhớ khái niệm *diện tích hình thang cong* thì sẽ tưởng tượng ra.)

Xem thêm [Precision-Recall-sikit-learn](#).

5.3. F1-score

F_1 score, hay F1-score, là *harmonic mean* của precision và recall (giả sử rằng hai đại lượng này khác không):

$$\frac{2}{F_1} = \frac{1}{\text{precision}} + \frac{1}{\text{recall}} \text{ hay } F_1 = 2 \frac{1}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

F_1 -score có giá trị nằm trong nửa khoảng $(0, 1]$. F_1 càng cao, bộ phân lớp càng tốt. Khi cả recall và precision đều bằng 1 (tốt nhất có thể), $F_1 = 1$. Khi cả recall và precision đều thấp, ví dụ bằng 0.1, $F_1 = 0.1$. Dưới đây là một vài ví dụ về F_1

Precision	Recall	F1F1
1	1	1
0.1	0.1	0.1
0.5	0.5	0.5
1	0.1	0.182
0.3	0.8	0.36

Như vậy, một bộ phân lớp với precision = recall = 0.5 tốt hơn một bộ phân lớp khác với precision = 0.3, recall = 0.8 theo cách đo này.

Trường hợp tổng quát của F_1 score là F_β score:

$$F_\beta = (1 + \beta^2) \frac{\text{precision} \cdot \text{recall}}{\beta^2 \cdot \text{precision} + \text{recall}}$$

F_1 chính là một trường hợp đặc biệt của F_β khi $\beta = 1$. Khi $\beta > 1$, recall được coi trọng hơn precision, khi $\beta < 1$, precision được coi trọng hơn. Hai đại lượng β thường được sử dụng là $\beta = 2$ và $\beta = 0.5$.

Trường hợp tổng quát của F1F1 score

là F_β score: $F_\beta = (1 + \beta^2) \frac{\text{precision} \cdot \text{recall}}{\beta^2 \cdot \text{precision} + \text{recall}}$

F_1 chính là một trường hợp đặc biệt của F_β khi $\beta = 1$. Khi $\beta > 1$, recall được coi trọng hơn precision, khi $\beta < 1$, precision được coi trọng hơn. Hai đại lượng β thường được sử dụng là $\beta = 2$ và $\beta = 0.5$.

5.4. Precision-recall cho bài toán phân lớp nhiều lớp

Cũng giống như ROC curve, precision-recall curve ban đầu được định nghĩa cho bài toán phân lớp nhị phân. Để có thể áp dụng các phép đo này cho bài toán multi-class classification, các đại lượng đầu ra (ground truth và predicted output) cần được đưa về dạng nhị phân.

Bằng trực giác, ta có thể đưa bài toán phân lớp nhiều lớp về bài toán phân lớp nhị phân bằng cách xem xét từng lớp. Với mỗi lớp, ta coi dữ liệu thuộc lớp đó có label là *positive*, tất cả các dữ liệu còn lại có label là *negative*. Sau đó, giá trị Precision, Recall, và PR curve được áp dụng lên từng lớp. Với mỗi lớp, ta sẽ nhận được một cặp giá trị precision và recall. Với các bài toán có ít lớp dữ liệu, ta có thể minh họa PR curve cho từng lớp trên cùng một đồ thị. Tuy nhiên, với các bài toán có rất nhiều lớp dữ liệu, việc này đôi khi không khả thi. Thay vào đó, hai phép đánh giá dựa trên Precision-Recall được sử dụng là *micro-average* và *macro-average*.

5.4.1. Micro-average

Xét ví dụ bài toán với 3 lớp dữ liệu, bộ phân lớp cho các tham số FP, TP, FN của mỗi lớp là:

```
tp1, fp1, fn1 = 10, 5, 3
tp2, fp2, fn2 = 17, 7, 10
tp3, fp3, fn3 = 25, 2, 4

from __future__ import print_function

def PR(tp, fp, fn):
    P = float(tp)/(tp + fp)
    R = float(tp)/(tp + fn)
    return (P, R)

(P1, R1) = PR(tp1, fp1, fn1)
(P2, R2) = PR(tp2, fp2, fn2)
(P3, R3) = PR(tp3, fp3, fn2)

print('(P1, R1) = (%.2f, %.2f)'%(P1, R1))
print('(P2, R2) = (%.2f, %.2f)'%(P2, R2))
print('(P3, R3) = (%.2f, %.2f)'%(P3, R3))

(P1, R1) = (0.67, 0.77)
(P2, R2) = (0.71, 0.63)
(P3, R3) = (0.93, 0.71)
```

$$\text{micro-average precision} = \frac{\sum_{c=1}^C \text{TP}_c}{\sum_{c=1}^C (\text{TP}_c + \text{FP}_c)} \quad \text{micro-average recall} = \frac{\sum_{c=1}^C \text{TP}_c}{\sum_{c=1}^C (\text{TP}_c + \text{FN}_c)}$$

với $\text{TP}_c, \text{FP}_c, \text{FN}_c$ lần lượt là TP, FP, FN của class c .

Micro-average precision và Micro-average recall đơn giản được tính bằng:

$$\text{precision} = \frac{\sum_{C=1}^C TP_C}{\sum_{C=1}^C (TP_C + FP_C)}$$
 micro-average

$$\text{recall} = \frac{\sum_{C=1}^C TP_C}{\sum_{C=1}^C (TP_C + FN_C)}$$
 micro-average

$$\text{precision} = \frac{\sum_{c=1}^C TP_c}{\sum_{c=1}^C (TP_c + FP_c)}$$
 micro-average

$$\text{recall} = \frac{\sum_{c=1}^C TP_c}{\sum_{c=1}^C (TP_c + FN_c)}$$
 với TP_c, FP_c, FN_c lần lượt là TP, FP, FN của class c .

Tức TP được tính là tổng của toàn bộ TP của mỗi lớp. Tương tự với FP và FN. Với ví dụ trên, micro-average precision và recall tính được là:

```
total_tp = tp1 + tp2 + tp3
total_fp = fp1 + fp2 + fp3
total_fn = fn1 + fn2 + fn3

micro_ap = float(total_tp)/(total_tp + total_fp)
micro_ar = float(total_tp)/(total_tp + total_fn)

print('(micro_ap, micro_ar) = (%.2f, %.2f)' % (micro_ap, micro_ar))

(micro_ap, micro_ar) = (0.79, 0.75)
```

Micro-average F-Score cũng được tính tương tự như F-score nhưng dựa trên micro-average precision và micro-average recall.

5.4.2. Macro-average

Macro-average precision là trung bình cộng của các precision theo class, tương tự với Macro-average recall. Với ví dụ trên, ta có

```
macro_ap = (P1 + P2 + P3)/3
macro_ar = (R1 + R2 + R3)/3

print('(macro_ap, macro_ar) = (%.2f, %.2f)' % (macro_ap, macro_ar))

(macro_ap, macro_ar) = (0.77, 0.70)
```

Macro-average F-Score cũng được tính tương tự như F-score nhưng dựa trên macro-average precision và macro-average recall.

6. Tóm tắt

- Accuracy là tỉ lệ giữa số điểm được phân loại đúng và tổng số điểm. Accuracy chỉ phù hợp với các bài toán mà kích thước các lớp dữ liệu là tương đối như nhau.
- Confusion matrix giúp có cái nhìn rõ hơn về việc các điểm dữ liệu được phân loại đúng/sai như thế nào.
- True Positive (TP): số lượng điểm của lớp *positive* được phân loại đúng là *positive*.
- True Negative (TN): số lượng điểm của lớp *negative* được phân loại đúng là *negative*.
- False Positive (FP): số lượng điểm của lớp *negative* bị phân loại nhầm thành *positive*.
- False Negative (FN): số lượng điểm của lớp *positive* bị phân loại nhầm thành *negative*.
- True positive rate (TPR), false negative rate (FNR), false positive rate (FPR), true negative rate (TNR):

	Predicted as Positive	Predicted as Negative	
Actual: Positive	TP	FN	$TPR = TP / (TP + FN)$ $FNR = FN / (TP + FN)$
Actual: Negative	FP	TN	$FPR = FP / (FP + TN)$ $TNR = TN / (FP + TN)$

- Khi kích thước các lớp dữ liệu là chênh lệch (*imbalanced data* hay *skew data*), precision và recall thường được sử dụng:

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

- F_1 score:

$$F_1 = 2 \frac{1}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

- Micro-average precision, micro-average recall:

$$\text{micro-average precision} = \frac{\sum_{c=1}^C TP_c}{\sum_{c=1}^C (TP_c + FP_c)} \quad \text{micro-average recall} = \frac{\sum_{c=1}^C TP_c}{\sum_{c=1}^C (TP_c + FN_c)}$$

với TP_c, FP_c, FN_c lần lượt là TP, FP, FN của class c .

- Micro-average precision, macro-average recall là trung bình cộng của các precision, recall cho từng lớp. Micro-average (macro-average) F1F1 scores cũng được tính dựa trên các micro-average (macro-average) precision, recall tương ứng.

LINK TƯƠNG ỨNG

<https://machinelearningcoban.com/2017/08/31/evaluation/>

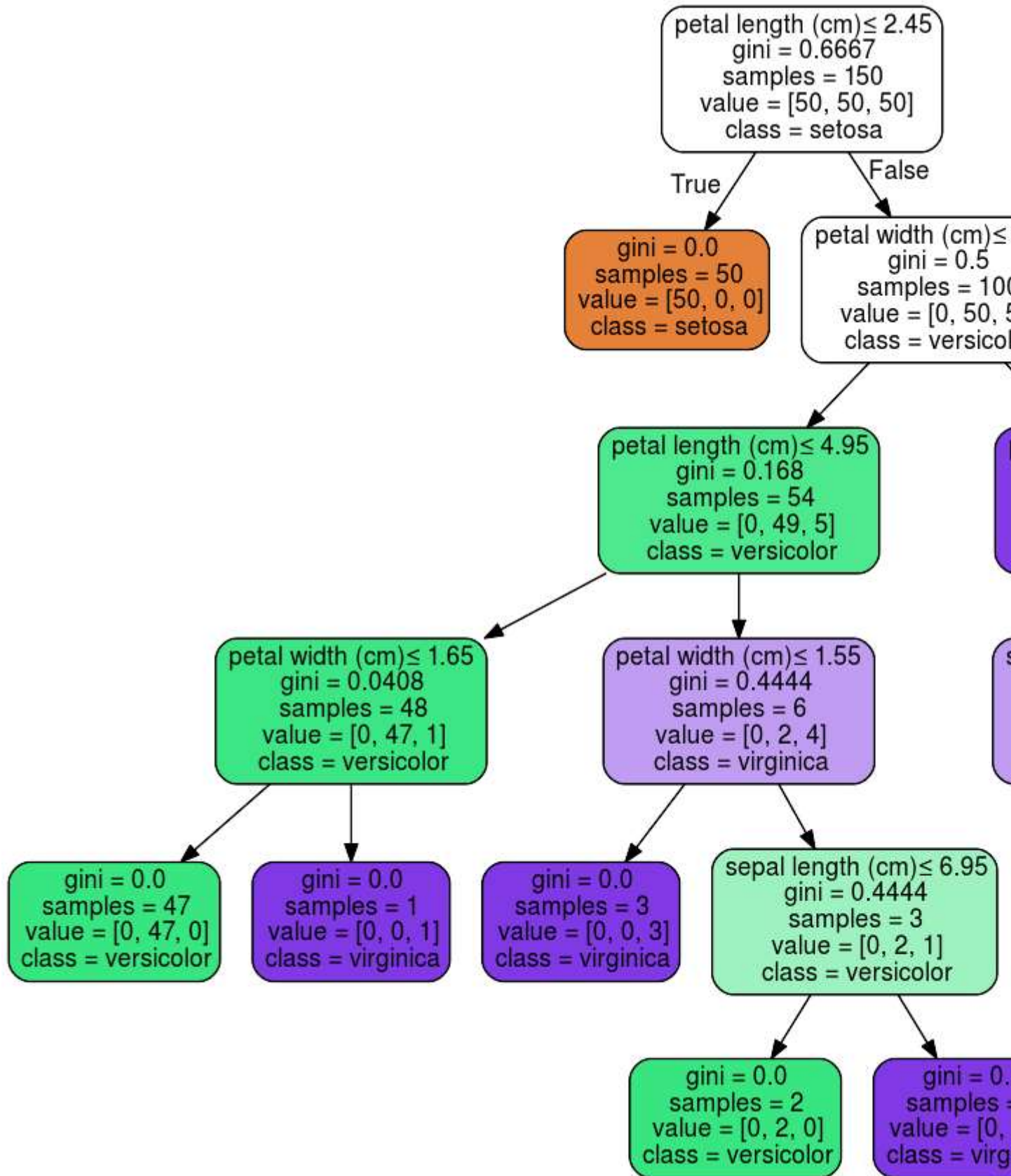
CÂY

Decision Tree và ý nghĩa của các chỉ số Gini Impurity và Entropy

09 October 2019

2371

[decision tree](#), [entropy](#), [gini](#), [machine learning](#)

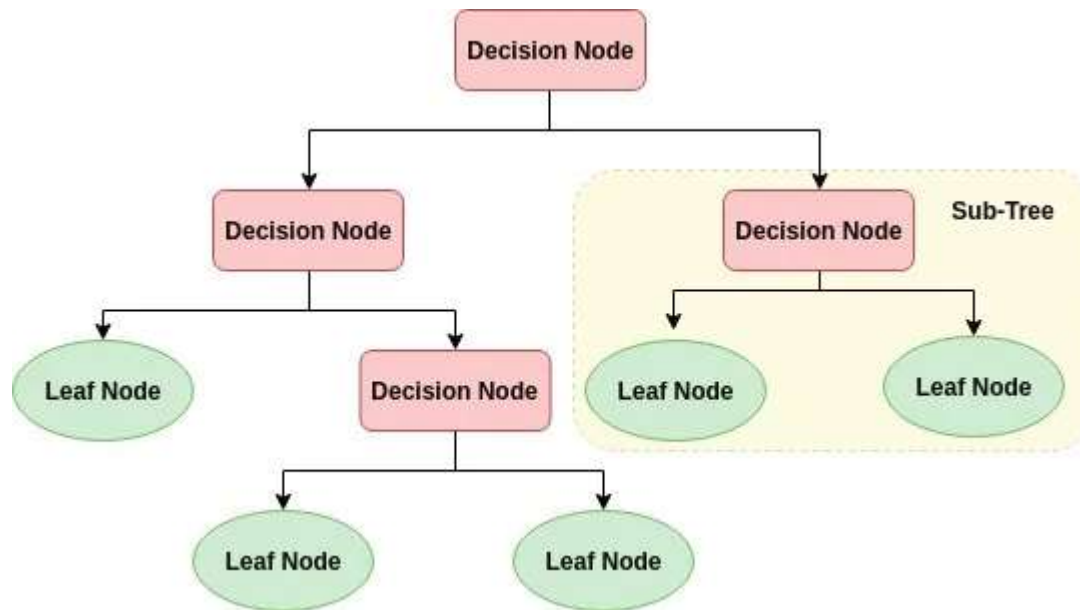


Overview về Decision Tree

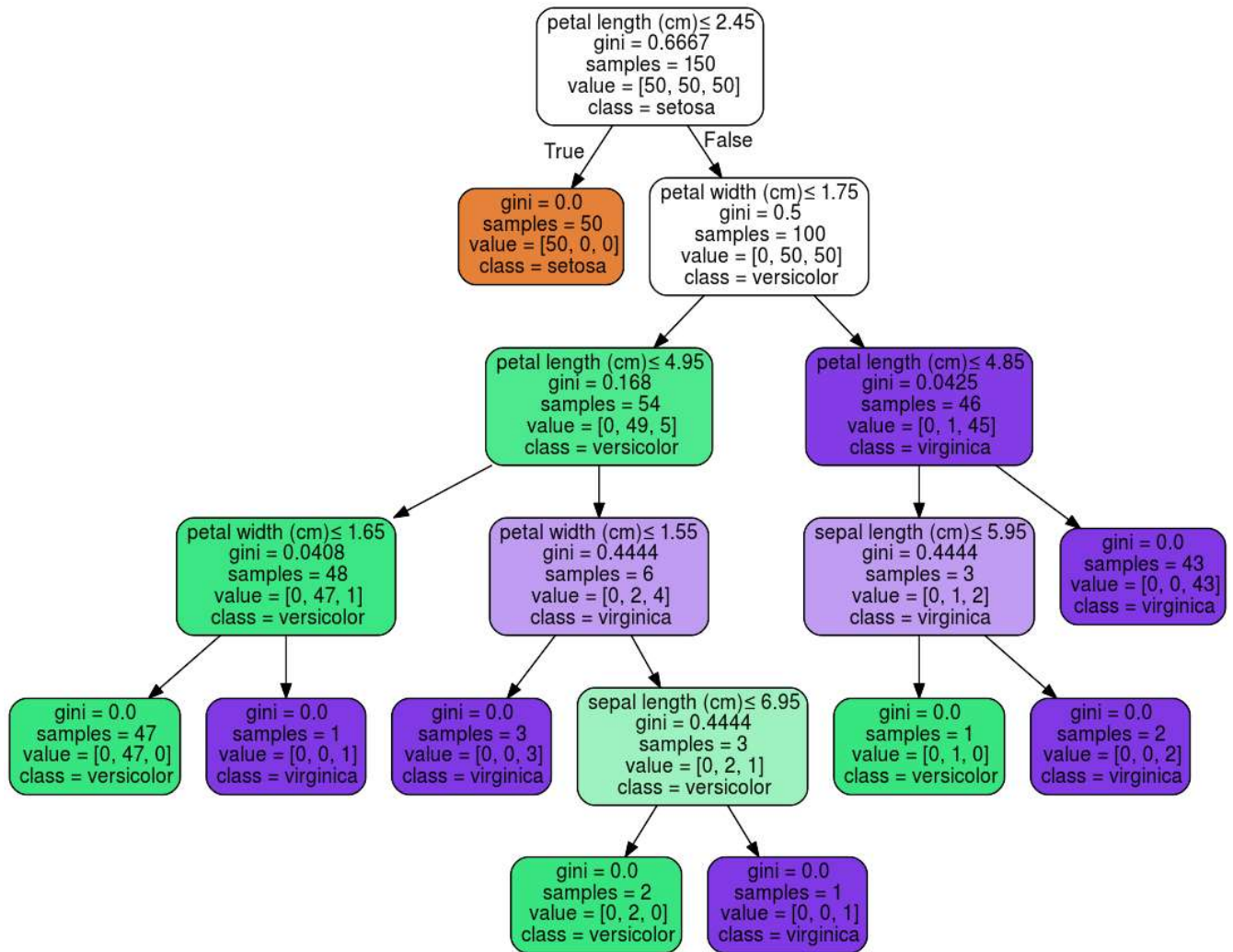
Thuật toán Decision Tree là một trong những thuật toán phân loại cơ bản nhất của Machine Learning và với việc sử dụng các thư viện hỗ trợ, ứng dụng Decision Tree trong thực tế trở nên vô cùng đơn giản, gói gọn trong vài dòng code. Ta có thể xem xét ví dụ sử dụng Python và thư viện scikit-learn ở dưới đây:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
iris = load_iris()
clf = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=17)
clf = clf.fit(iris.data, iris.target)
```

Chỉ với 5 dòng code, một model DecisionTree đã được train để phân loại với tập data Iris có sẵn của scikit-learn. DecisionTree là một whitebox model, ta có thể dễ dàng nhìn được cấu trúc dữ liệu bên trong và logic để phân loại data của nó. Có thể hình dung cấu trúc dữ liệu của DecisionTree như sau.



Decision Node thể hiện một rule để phân nhánh data theo một thuộc tính (có thể hiểu là một rule áp dụng lên feature của data), mỗi nhánh từ Decision Node là kết quả phân nhánh theo rule đó, Leaf Node là kết quả phân loại cuối cùng. Ta có thể hình dung rõ ràng hơn với graph thể hiện cụ thể một model Decision Tree đã được train ở dưới đây:



Để xây dựng cấu trúc cây ở trên, thuật toán Decision Tree đơn giản sẽ bao gồm các bước sau:

- Chọn lựa thuộc tính của data để chia data sử dụng Attribute Selection Measures(ASM: Chỉ số đánh giá lựa chọn thuộc tính)
- Tạo descision node với feature và điều kiện ở trên.
- Phân nhánh data tạo các child node và lặp lại tiến trình ở trên cho đến khi một trong các điều kiện sau thoả mãn, ta sẽ có leaf node:
 - Tất cả data của node đều thoả mãn điều kiện của decision node.
 - Không có feature với điều kiện nào có thể được chọn nữa.
 - Không còn data nào thoả mãn điều kiện của decision node.

Bài viết hôm nay sẽ tập trung giới thiệu về các ASM, chỉ số đánh giá thuộc tính. Bằng việc phân tích dữ liệu đầu vào, ASM đưa ra ranking cho các thuộc tính của data, sau đó

lựa chọn thuộc tính tốt nhất làm điểm để chia nhánh dữ liệu. Nếu sử dụng scikit-learn thì chỉ số này có thể được cài đặt bằng cách thay đổi giá trị của biến **criterion** như ví dụ code ở trên. Scikit-learn hỗ trợ 2 giá trị cho biến **criterion** là **gini** và **entropy** tương ứng với hai chỉ số là **gini impurity** và **information gain**.

Entropy và information gain

Entropy là một khái niệm khá thông dụng được dùng trong nhiều lĩnh vực vật lý, toán học v.v.. Trong lĩnh vực xử lý thông tin, nhà toán học Claude Shannon đã đưa ra khái niệm **information entropy** (Shannon entropy). Shannon entropy thể hiện mức độ hỗn loạn hay độ nhiễu của data. Với một hệ thống với N trạng thái có thể xảy ra thì công thức của Shannon entropy sẽ như ở dưới:

$$S = - \sum_{i=1}^N p_i \log_2 p_i$$

Trong đó p_i là xác suất trạng thái thứ i xuất hiện ở trong hệ thống. Giá trị entropy càng cao thì độ hỗn loạn của hệ thống càng cao, còn nếu càng thấp thì hệ thống càng có trật tự. Và một hệ thống có trật tự cũng tương đương với việc data được phân nhánh một cách chuẩn chỉ.

Ta có thể xem xét một ví dụ đơn giản. Có 20 viên bi, được xếp theo thứ tự như sau, có 9 viên màu xanh, 11 viên màu đỏ (o là màu xanh, x là màu đỏ).

```
x o o o o x x x x o o o o x x x x x x o
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
```

Bài toán là dự đoán màu của viên bi dựa vào thứ tự sắp xếp. Nếu không để ý tới thứ tự thì xác suất ta lấy được viên màu xanh là $p_1 = \frac{9}{20}$, màu đỏ là $p_2 = \frac{11}{20}$. Entropy ban đầu của tập data sẽ có giá trị là $S_0 = -\frac{9}{20} \log_2 \frac{9}{20} - \frac{11}{20} \log_2 \frac{11}{20} \approx 1$.

Nếu ta tiếp tục chia tập các viên bi thành 2 nhóm khác nhau theo tiêu chí chia là thứ tự nhỏ hơn hoặc bằng 13 là 1 nhóm, các viên bi còn lại vào một nhóm:

Bài toán là dự đoán màu của viên bi dựa vào thứ tự sắp xếp. Nếu không để ý tới thứ tự thì xác suất ta lấy được viên màu xanh là $p_1 = \frac{9}{20}$, màu đỏ là $p_2 = \frac{11}{20}$. Entropy ban đầu của tập data sẽ có giá trị là $S_0 = -\frac{9}{20} \log_2 \frac{9}{20} - \frac{11}{20} \log_2 \frac{11}{20} \approx 1$.
Nếu ta tiếp tục chia tập các viên bi thành 2 nhóm khác nhau theo tiêu chí chia là thứ tự nhỏ hơn hoặc bằng 13 là 1 nhóm, các viên bi còn lại vào một nhóm:

```
x o o o o x x x x o o o o x x x x x x o
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
```



```

[ ]
[ x <= 13 ]
[ / \ ]
X O O O O X X X X O O O O      X X X X X X X O
1 2 3 4 5 6 7 8 9 10 11 12 13      14 15 16 17 18 19 20

```

Nhóm bên trái có 8 viên xanh và 5 viên đỏ nên nó giá trị entropy $S_1 = -\frac{5}{13}\log_2 \frac{5}{13} - \frac{8}{13}\log_2 \frac{8}{13} \approx 0.96$.

Nhóm bên phải có 1 viên xanh và 5 viên đỏ nên nó có giá trị entropy $S_2 = -\frac{1}{7}\log_2 \frac{1}{7} - \frac{6}{7}\log_2 \frac{6}{7} \approx 0.6$.

Ta nhận thấy entropy của mỗi nhóm sau khi chia đều giảm so với entropy ban đầu của hệ thống. Giá trị entropy thể hiện mức độ hỗn loạn của hệ thống, do đó khi entropy giảm hệ thống có trật tự hơn hay có thể nói có nhiều thông tin hơn. Do vậy, độ giảm của entropy được gọi là **information gain** và có công thức như sau:

$$IG(Q) = S_O - \sum_{i=1}^q \frac{N_i}{N} S_i$$

Nhóm bên trái có 8 viên xanh và 5 viên đỏ nên nó giá trị entropy $S_1 = -\frac{5}{13}\log_2 \frac{5}{13} - \frac{8}{13}\log_2 \frac{8}{13} \approx 0.96$.

Nhóm bên phải có 1 viên xanh và 5 viên đỏ nên nó có giá trị entropy $S_2 = -\frac{1}{7}\log_2 \frac{1}{7} - \frac{6}{7}\log_2 \frac{6}{7} \approx 0.6$.

Ta nhận thấy entropy của mỗi nhóm sau khi chia đều giảm so với entropy ban đầu của hệ thống. Giá trị entropy thể hiện mức độ hỗn loạn của hệ thống, do đó khi entropy giảm hệ thống có trật tự hơn hay có thể nói có nhiều thông tin hơn. Do vậy, độ giảm của entropy được gọi là **information gain** và có công thức như sau:

Trong đó:

IG là giá trị information gain

Q là điều kiện để chia data

q là số nhóm sau khi chia

Ni là số lượng data của mỗi nhóm

Áp dụng với ví dụ các viên bi ở trên, information gain sau khi chia thành 2 nhóm với điều kiện thứ tự $x \leq 13$ sẽ là:

$$IG(x \leq 13) = S_0 - \frac{13}{20}S_1 - \frac{7}{20}S_2 \approx 0.16.$$

Nếu ta tiếp tục chia nhánh nhóm các viên bi, cho tới khi mỗi nhóm chỉ có duy nhất một loại bi thì lúc đó entropy của các nhóm này sẽ đều là 0 (Do $\log_2[1]=0$). Vậy là ta đã có được các Leaf Node và một Decision Tree dùng để dự đoán màu của viên bi theo thứ tự, dù là model này đã overfit cứng với tập viên bi được cung cấp.

Trong ví dụ ở đây, ta chọn điều kiện để chia nhóm các viên bi một cách ngẫu nhiên, nhưng trong thực tế khi training Decision Tree, điều kiện chia sẽ được chọn sao cho giá trị information gain sau khi chia là lớn nhất.

Gini Impurity

Đơn giản hơn so với Entropy và information gain, Gini Impurity là chỉ số thể hiện mức độ phân loại sai khi ta chọn ngẫu nhiên một phần tử từ tập data. Gini Impurity có công thức như sau:

$$G = \sum_k p_k(1-p_k) = 1 - \sum_k (p_k)^2$$

Trong đó:

G là giá trị Gini Impurity

k số các lớp có trong tập data

p_k là xác suất mà một phần tử ngẫu nhiên thuộc lớp k

Quay lại ví dụ chọn bi ở phần trên, Gini impurity ban đầu của tập data sẽ là $G_0 = 1 - (9/20)^2 - (11/20)^2 \approx 0.5$

Cho nhóm 1: $G_1 = 1 - (5/13)^2 - (8/13)^2 \approx 0.47$

Cho nhóm 2: $G_2 = 1 - (1/7)^2 - (6/7)^2 \approx 0.24$

$$G = \sum_k p_k * (1 - p_k)^2 = 1 - \sum_k (p_k)^2$$

Trong đó:

G là giá trị Gini Impurity

k số các lớp có trong tập data

p_k là xác suất mà một phần tử ngẫu nhiên thuộc lớp k

Quay lại ví dụ chọn bi ở phần trên, Gini impurity ban đầu của tập data sẽ là $G_0 = 1 - (\frac{9}{20})^2 - (\frac{11}{20})^2 \approx 0.5$

Cho nhóm 1: $G_1 = 1 - (\frac{5}{13})^2 - (\frac{8}{13})^2 \approx 0.47$

Cho nhóm 2: $G_2 = 1 - (\frac{1}{7})^2 - (\frac{6}{7})^2 \approx 0.24$

Ta thấy, giá trị Gini Impurity sau khi chia của mỗi nhóm đều nhỏ hơn so với giá trị ban đầu $\Rightarrow$ Sau khi chia nhóm mức độ phân loại sai của hệ thống đã giảm. Độ giảm của Gini Impurity được gọi là **Gini Gain** và có công thức tính tương tự như information gain, chỉ khác là ta sẽ sử dụng giá trị Gini Impurity thay vì Entropy:

$$GG(Q) = G_0 - \sum_{i=1}^q \frac{N_i}{N} G_i$$

Với ví dụ chọn bi, Gini Gain sau khi chia nhóm lần đầu sẽ là:

$$GG(x \leq 13) = G_0 - \frac{13}{20} G_1 - \frac{7}{20} G_2 \approx 0.11$$

$$GG(Q) = G_0 - \sum_{i=1}^q \frac{N_i}{N} G_i$$

Với ví dụ chọn bi, Gini Gain sau khi chia nhóm lần đầu sẽ là:

$$GG(x \leq 13) = G_0 - \frac{13}{20} G_1 - \frac{7}{20} G_2 \approx 0.11$$

Cũng tương tự như với khi sử dụng Entropy, Gini Impurity của một Leaf Node sẽ là 0 và khi training Decision Tree, điều kiện được chọn để chia nhóm cũng sẽ được chọn sao cho giá trị Gini Gain là lớn nhất

Kết luận

Trong thực tế qua kiểm nghiệm thì việc chọn Entropy hay Gini Impurity cũng sẽ không ảnh hưởng nhiều đến kết quả dự đoán của Decision Tree. Do đó khi tuning parameter, ta

có thể bỏ qua việc tuning parameter này.

Nhưng dù sao thì hi vọng bài viết đã đem lại cho người đọc thêm kiến thức để hiểu rõ hơn về Decision Tree cũng với các chỉ số phân chia nhánh xây dựng Tree của nó.

- <https://mlcourse.ai/articles/topic3-dt-knn/>
- <https://victorzhou.com/blog/gini-impurity/>
- <https://www.datacamp.com/community/tutorials/decision-tree-classification-python>
- https://en.wikipedia.org/wiki/Decision_tree_learning